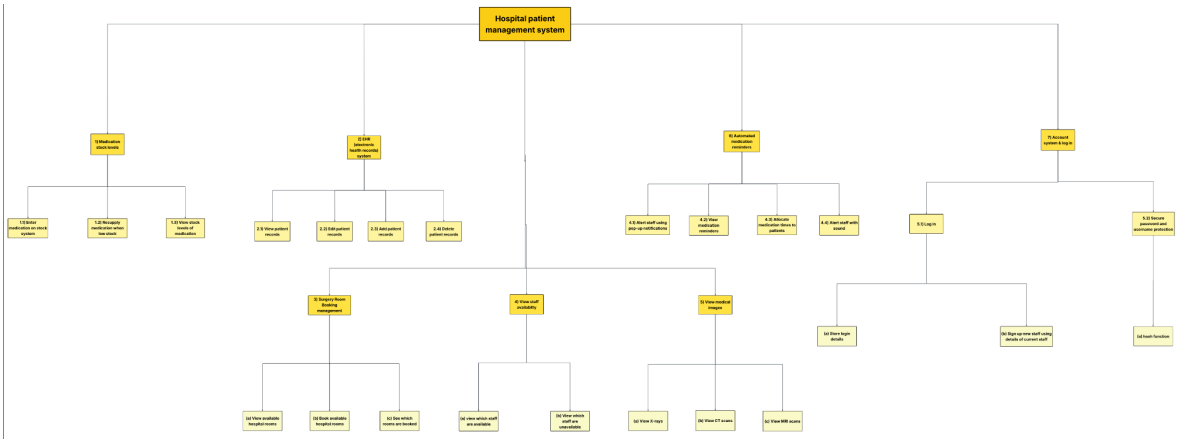
NEA Design - Hospital management system

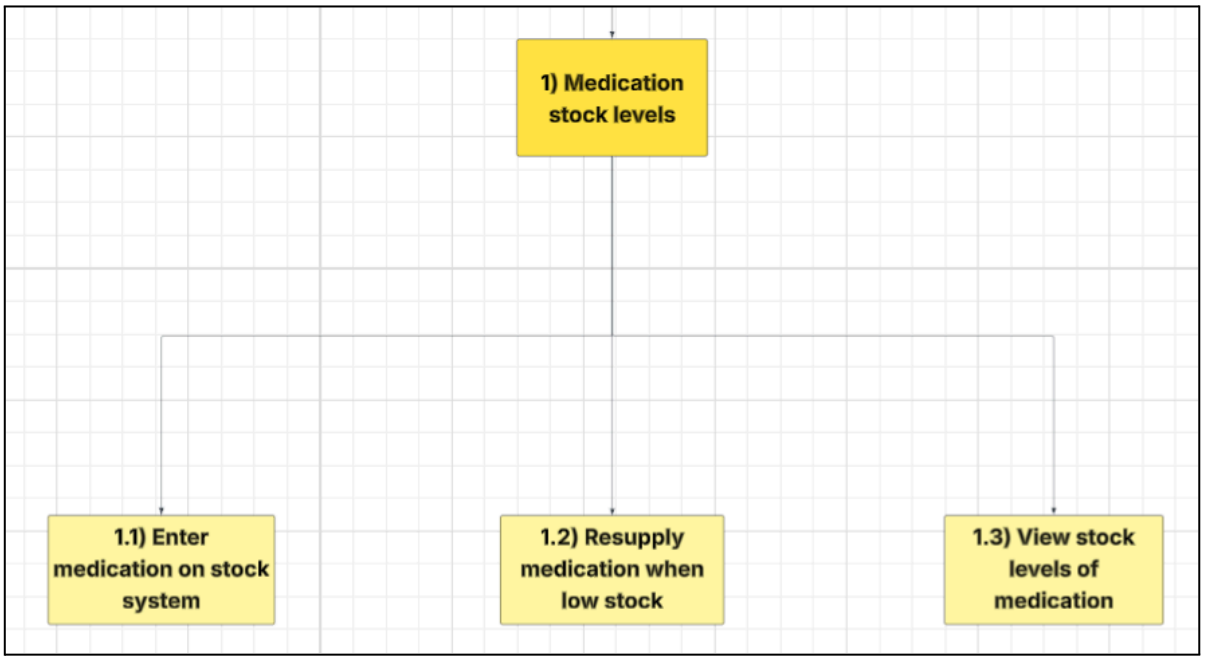
3.2.1 Decompose the problem:

3.2.1 a) What are the smaller sub-modules of the solution



Starting from left to right!

1) Medication stock levels:

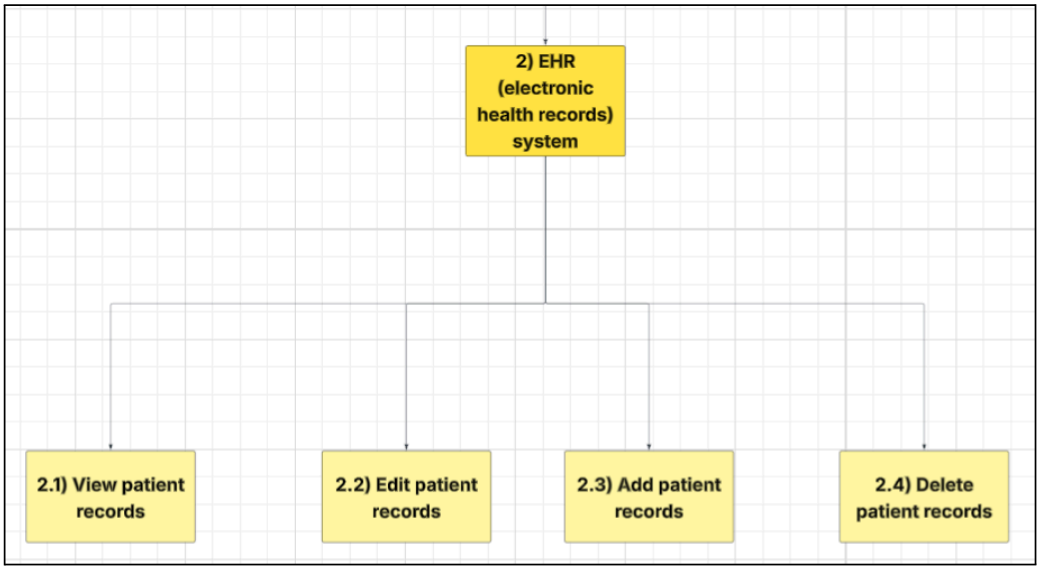


This module of the system will allow staff to handle and view stock levels of medication by entering the medication details on the system, such as the medication name, purpose and what it is used for (e.g. painkillers, antibiotics, etc), what symptoms and diseases it treats, startup stock (e.g. how much stock is in each medication so that each medication on the system has different levels of stock). The staff will also be able to

view the stock levels of the medication and resupply medication when certain ones are low.

Sub-module	Explanation
1.1) Enter medication on the stock system	To allow staff to be alerted about medication times, they must enter medication and medication details into the system and assign medication to patients before they can be alerted by the medication times of those patients. Staff can enter details such as: • Medication name • Purpose & use (e.g. painkillers, antibiotics, etc) • What symptoms & diseases does it treat
1.2) Resupply medication when stock is low	Staff also have to ensure that they have sufficient medication to treat all of their patients. This sub-module will allow staff to resupply their medication when they're close to running out, and so every time an alert occurs, the assigned medication of that alert is reduced by 1 each time. This will work by a display telling staff that medication is low, with a button to resupply for that medication.
1.3) View stock levels of medication	This will allow staff to view how much stock they have for certain medicines in their inventory. This allows them to track the actual supply level of each medicine registered in their system. I am planning to do this using a stock meter in the form of a green bar (e.g. a full green health bar indicates max stock level, whereas a partially full green bar may suggest the need for resupplying). I will also present this data using numerical data/fractions, e.g. 9/1,5 where 15 is the maximum stock level for any medication. I will also make it so that each time an alert for a certain medication happens, the stock count for that certain medication decreases by 1 each time.

2) EHR (Electronic Health Records) system:

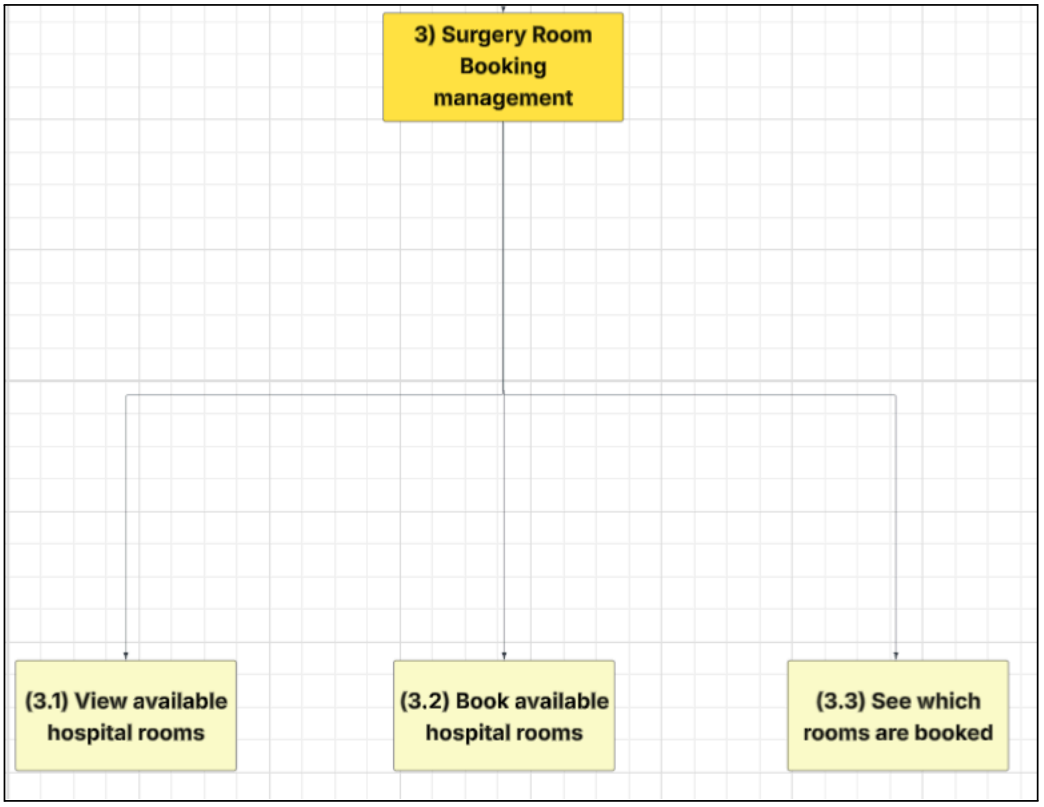


This module will allow staff to add and delete patient records as well as alter, change and update patient information within the records themselves. This allows hospitals and staff to abide by the principles of the Data Protection Act, where data about patients and individuals must be processed fairly and legally, as well as keep up-to-date information about their patients. This is also because EHRs in hospitals hold lots of sensitive data of patients, e.g.:

- Patient's name/legal name
- Date of birth
- Contact information (e.g. Email addresses, phone numbers, etc).
- Legal information (e.g. residency/settlement status, nationality, etc).
- Medical history & information (e.g. current diseases, symptoms, treatment/medication)
- Potential graphical information (e.g. X-ray, MRI scans, etc).

Sub-module	Explanation
2.1) View patient records	Before patient data can be altered, staff have to check if there are any errors or obsolete data in the records. This sub-module allows staff to access and view patient records and information for any potential errors or misinformation.
2.2) Alter patient records	Now that staff are able to check for errors, this means that they are now able to change them to present the correct information. This sub-module allows staff to change obsolete data into correct, accurate and up-to-date information.
2.3) Add patient records	This sub-module allows staff to enter new patients onto the system, as well as their data and information. This is because hospitals regularly encounter new patients and need to adapt by adding new records for those new patients.
2.4) Delete patient records	This sub-module allows staff to delete patient records and information. This sub-module ensures that staff are able to abide by one of the key principles of the Data Protection Act (DPA), where sensitive data and information should not be held for longer than required.

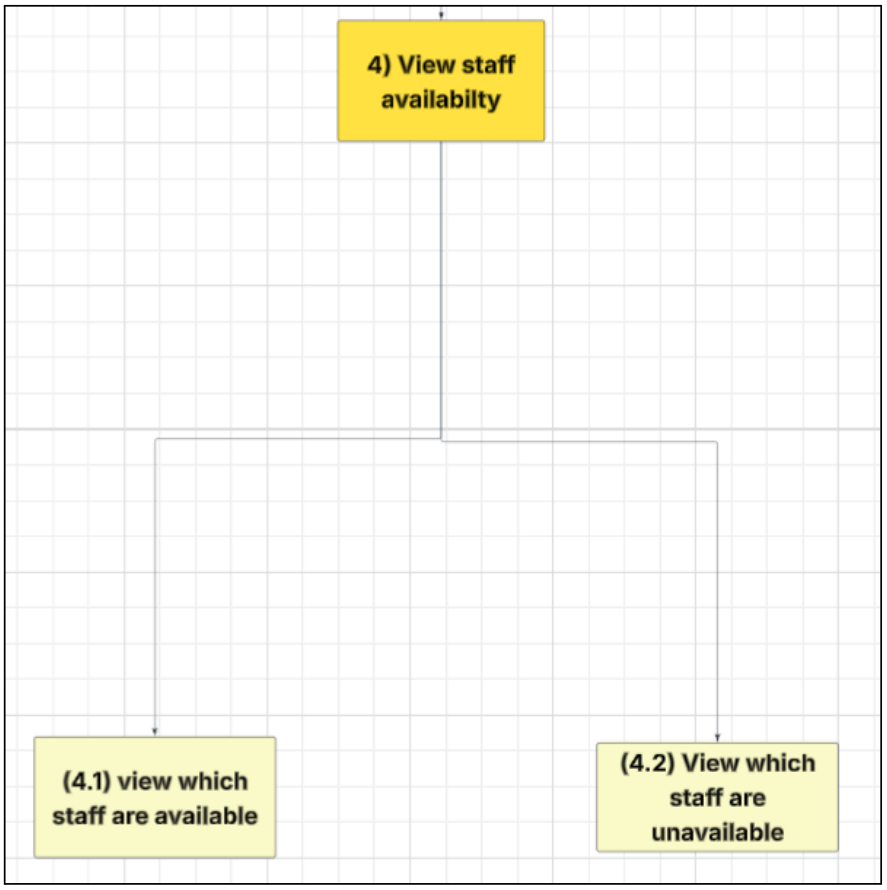
3) Surgery room booking management



This module is in charge of managing rooms in a hospital. This allows staff to view available hospital rooms and book them for a set duration of time. This module will also allow staff to see which rooms are booked or not to avoid clashes between staff.

Sub-module	Explanation
(3.1) View available hospital rooms	Before surgery-rooms can be booked by nurses and staff, they need to see if they're unbooked or booked by different staff. This will also show you which staff member booked the corresponding surgery room, and for how long the room is booked for..
(3.2) Book available hospital rooms	This allows staff members to book hospital rooms that are not yet taken/booked by other staff members. This will also allow staff to choose the period of time for which the room is booked when they are planning to book the room. **I will also be planning to add a feature that will allow staff to choose the duration of time for which the room is booked, using sensible limits (e.g. from 0 hrs & 30 minutes to 72 hrs, with hrs representing minutes and minutes representing seconds, i.e. 30 hrs = 30 minutes in real time)**
(3.3) See which rooms are booked	This allows staff to avoid booking the same rooms for different patients twice, so that each room only holds one patient. This helps avoid conflict when booking surgery rooms. This also allows them to see which staff member booked those rooms and for how long.

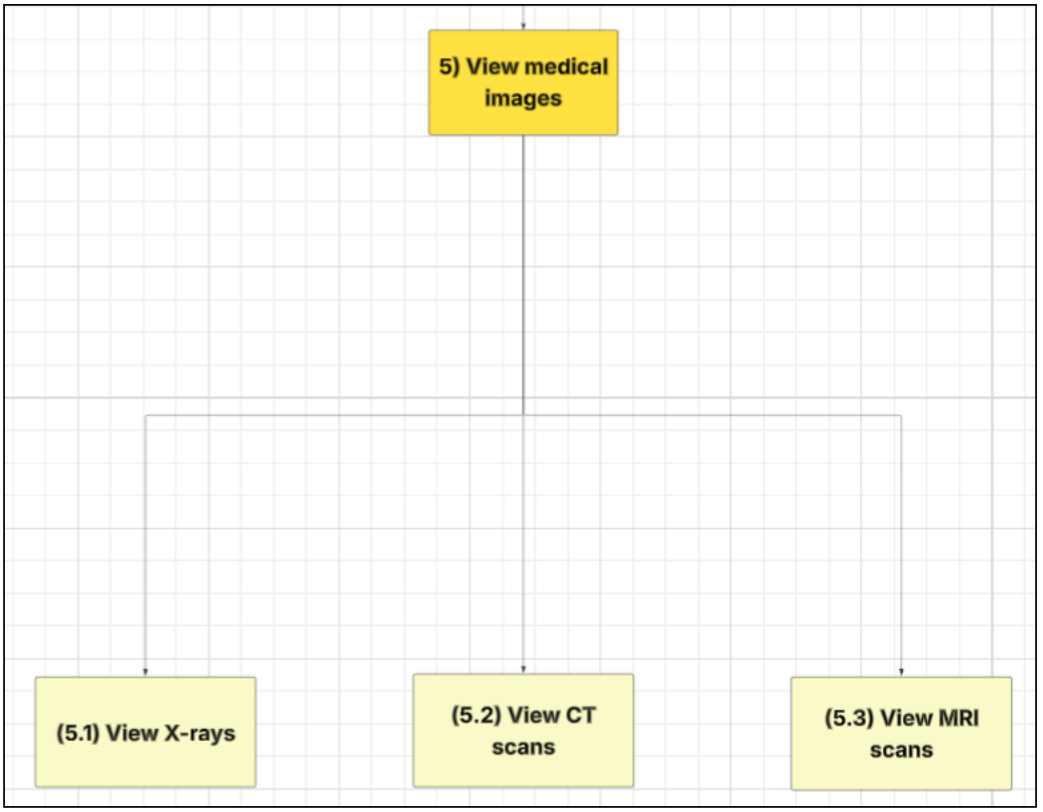
4) View staff availability



When staff are servicing a patient, they are displayed as busy, on the system. This module helps staff members see which colleagues are busy or available, on the system. This is to ensure that staff members are not overworked with duties.

Sub-module	Explanation
(4.1) View which staff are available	Since staff (e.g. nurses, surgeons, pharmacists, health professionals) are usually busy serving & servicing patients in rooms, this sub-module will allow staff to see if staff are available, i.e. outside of room-booking times, when the duration of the booking period expires (when staff choose how long rooms are booked for).
(4.2) View which staff are unavailable	This sub-module ensures that staff are not overworked by seeing if they currently have responsibilities/duties they need to fulfil. Therefore, being able to view unavailable staff allows the staff using the system to be informed on whether staff are currently busy or not.

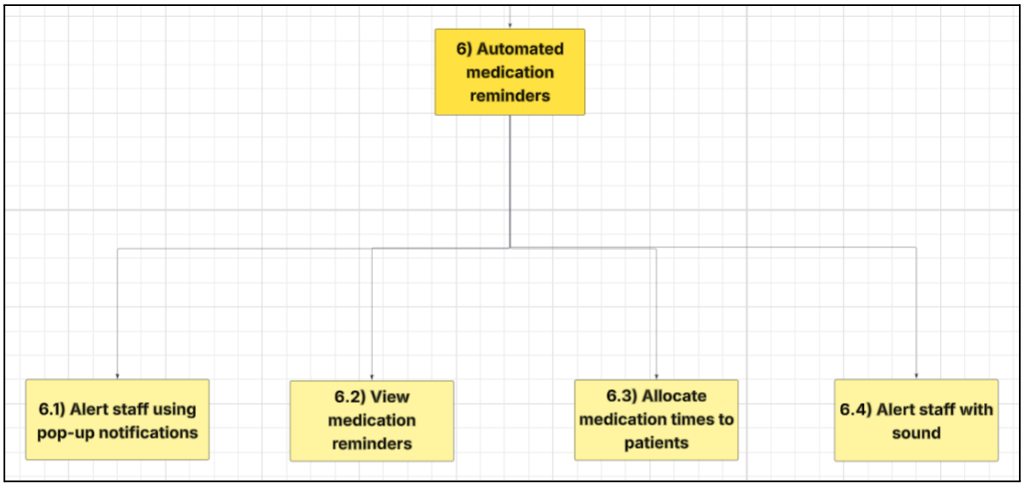
5) View medical images



This sub-module allows staff to view medical images of patients, such as patient injuries. This module includes three types of medical screening/imaging; x-ray, CT, and MRI scans. Each form of medical scanning has its own uses and functions:

Sub-module	Explanation
(5.1) View X-ray scans	This sub-module allows staff to create images of the inside of the body, primarily used for diagnosing conditions like bone fractures and joint problems. This can also help to detect internal issues such as lung infections or problems with internal organs. Used to diagnose conditions such as dislocations, misalignments and other fractures.
(5.2) View CT scans	CT (Computed Tomography) scans allow staff to provide a higher level of detail, showing the patient's injuries by being able to view the injuries from different angles. It does this by combining multiple 2D x-ray images from different angles to create a single interactive 3D image, used to diagnose conditions such as bone fractures, blood clots and organ injuries.
(5.3) View MRI scans	MRI (Magnetic Resonance Imaging) allows staff to view highly detailed 2D images, showcasing soft tissue, organs and bone ligaments. MRI is useful for detecting issues like inflammation, tumours, ligament tears and other diseases.

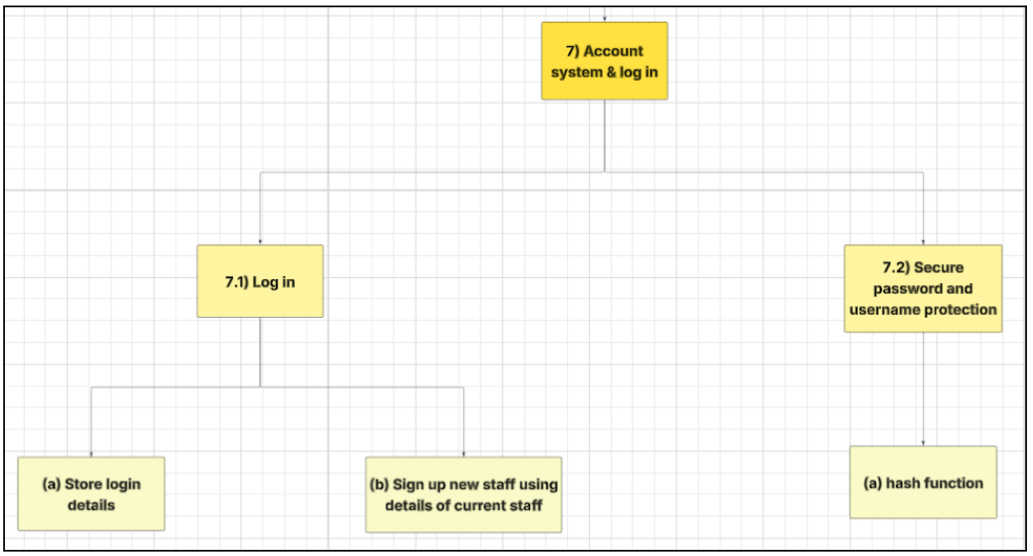
6) Automated medication reminders



This module is in charge of alerting staff about the medication of patients, using sound alerts and screen pop-ups. This module will also allow staff to view info on medication times (e.g. what time, patient, and medicine distributed to that patient), as well as allocate medication times to patients as inputs for the module.

Sub-module	Explanation
6.1) Alert staff using pop-up notification	Having this sub-component means that staff are provided with the full context and are fully aware of the medication times. This is because sound alerts lack context on the task to be completed (distributing medicine to the patient). Therefore, this sub-module helps avoid confusion, especially for staff with hearing difficulties
6.2) View medication reminders	This sub-module will allow staff to see all patient medicine alerts across the whole hospital in real-time. This is because, as alerts take place, they are instantly removed from the system after the medicine has been distributed
6.3) Allocate medication times to patients	Before staff can be alerted about medication reminders, there needs to be some input for that to happen. In summary, this sub-module allows staff to assign registered medicines on the system to certain patients and records.
6.4) Alert staff with sound	This sub-module allows staff with visual difficulties and eye-impairments to be informed about the reminders of patients. The addition of this sub-module and the pop-up notifications ensures that all staff are able to be informed of medicine times. Additionally, sound alerts also reduce the chance of staff accidentally missing the reminder.

7)Account system & log in



This module of the system is in charge of the log in and security aspects of the hospital patient software. Not only does this allow new staff to login to the system using the details of an existing staff member, It also validates the staff log in details (username and password), using a hash function.

7.1) Log in

Sub-module	Explanation
(a) Store login details	Since staff members regularly sign out so that different staff can use the same computer, storing the login details for different staff members allows different nurses, doctors, physicians & other staff to log in and use the system's features.
(b) Sign up new staff using the details of current staff	Allowing new staff members to sign up using the username and password of existing staff members helps reduce the chances of unauthorised system access, as new staff members will need to physically ask other staff members for their login details. So requires virtual and physical validation to authorise new entrants to the system.

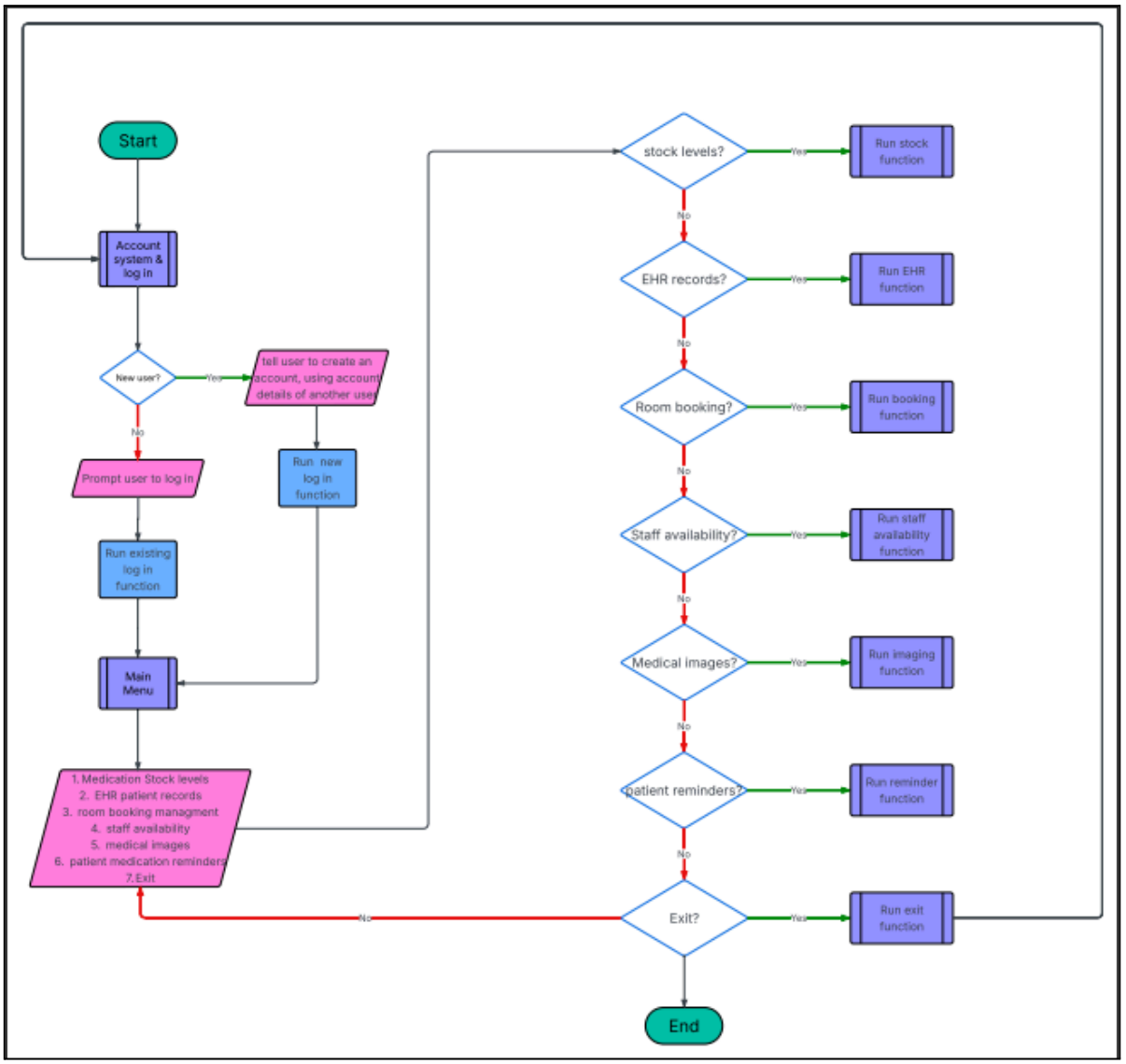
7.2) Secure password and username protection

Sub-module	Explanation
(a) Hash function/table	This will be the most important sub-module of the system. This ensures that staff usernames and passwords are kept highly secure, compared to storing the passwords in plain text. This is because, as staff enter login details, this module does not require the stored hash values to be converted back into plain text, making it one of the most secure methods of storing sensitive patient information.
(b) adding a salt to the end of a password	A salt is a short and randomly generated string of characters added at the end of a password, to ensure that two users with the same password have two different final passwords and so, produce

3.2.2 Describe the solution:

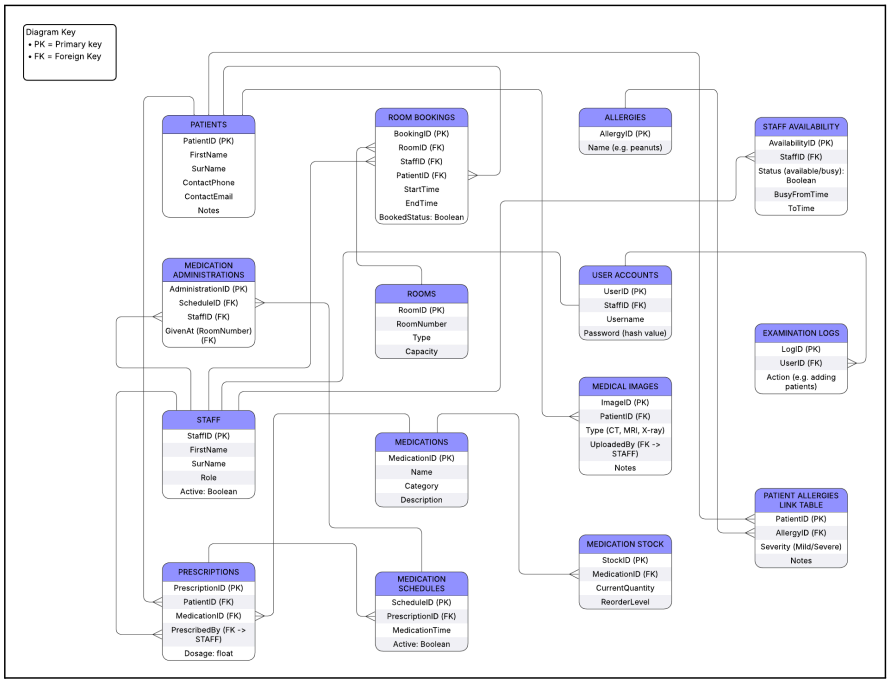
3.2.2 a) Explain and justify the structure of the solution

System flowchart:
The program will have many sequential steps needed to give outputs back to the user. Additionally, the program must be as user-friendly as possible for all hospital staff to use the system hassle-free. Below, I have constructed a system flowchart of the main menu, so that I can visualise each step of the process.



Entity-relationship diagram:

Aside from the sequencing and output of the system, there will also be a range of relational databases that my system will hold for various database information (e.g. staff, patients, medications, bookings, allergies, images, etc.). I have constructed an Entity-Relationship diagram, showing all of my possible databases and their relationships between them.



<u>IPSO diagrams</u>

An IPSO (Input, Process, Storage, Output), highlights all of the key steps each feature of the system goes through, from input given by the user, to how that input is processed & given back to the user

As output. I have constructed an IPSO table below, highlighting all of these steps, concerning all of the features of the system

1) Create patient records

INPUT	PROCESS	STORAGE	OUTPUT
FirstName, Surname, DOB, ContactPhone, ContactEmail, notes	Validate presence of characters (e.g. name not empty), make a patient row with the new patientID	Insert row into PATIENTS table	Message to show that the patient record has been made, e.g. “Patient Created” message

Justification: This will be the main EHR function. This is because the process of validating the patient records as they're being created (e.g., make sure that name and surname fields aren't left empty) will reduce the chance of data entry errors during testing.

2) Edit/delete patient records

INPUT	PROCESS	STORAGE	OUTPUT
PatientID, edited fields e.g., Name, DOB, etc.	Lookup PatientID in the PATIENTS table, re-validate edited fields.	update/delete in PATIENTS table.	Message to show patient records have been updated e.g. "successfully updated/deleted record"

Justification: Editing and deletion of records are handled through carefully-controlled updates to reduce accidental data loss. By referencing patients through their ID, the system ensures that only the correct record is changed, and the rest remain unchanged.

3) Upload medical image (x-ray/CT/MRI)

INPUT	PROCESS	STORAGE	OUTPUT
PatientID, Image file, Type (X-ray, CT, MRI), UploadedBy(StaffID)	Check file type & file size, generate imageID, link to patient	Save file (desktop), Insert into MEDICAL_IMAGES row.	Message e.g., "Upload successful"

Justification: Storing image metadata separately from the actual file allows the system to retrieve the image while keeping the database lightweight. Linking each image to a patient ID also ensures that records always display the correct image.

4) Create a prescription

INPUT	PROCESS	STORAGE	OUTPUT
PatientID, MedicationID, Dosage, medication times, Prescribed by (StaffID), Start/End dates	Validate dates & dosage, generate PrescriptionID	Insert into PRESCRIPTIONS table/row	Prescription summary, message "Create schedule"

Justification: This process allows staff to create prescriptions independently of scheduling or stock control. Additionally, this allows staff to validate dosage, medication type, and prescription duration before linking it to a patient.

5) Build a Medication schedule

INPUT	PROCESS	STORAGE	OUTPUT
PrescriptionID, dose times, repeat rule (e.g. every 8 hours)	Expand to next due event (no pre-generation of infinite events);	Insert into MEDICATION _SCHEDULES row	NextDue medication/dose time shown on patient timeline.

	compute NextDueAt	(PrescriptionID, NextDueAt, RepeatRule)	Message “Schedule saved”

Justification: Generating schedules from existing prescriptions ensures each dosage event is calculated accurately & avoids human error in the timings of the patients' doses. This approach supports automation (reminders and sound alerts), and so, staff can be busy on other tasks in the meantime.

6) Alert & Administration Acknowledgement

INPUT	PROCESS	STORAGE	OUTPUT
User clicks “acknowledge” during a sound alert for a dose time of a certain patient, StaffID	Pop-up + sound when NextDueAt = Now, or when NextDoseTime = now. On acknowledge, create administration record, and decrement MEDICATION_STOCK for that MedicationID.	Insert into MEDICATION_ADMINISTRATIONS table/row (ScheduleID, AdministeredAt, AdministeredBy), update MEDICATION_SCHEDULES,	Pop-up closes, updated counts (stock meter for that medicine) -> (smaller green stock bar)

Justification: By triggering alerts based on real-time schedule times, the system ensures critical medication times are not missed. The acknowledgement process automatically updates the stock count, improving patient safety and healthcare/medication delivery

7) Room booking (Conflict detection)

INPUT	PROCESS	STORAGE	OUTPUT
RoomID, StaffID, optional PatientID, StartTime, EndTime, Purpose	Validate times, check for query overlaps on RoomID between users/staff, create BookingID	Insert into ROOM_BOOKINGS table/row	Message “Booking is complete” with details, e.g. start/end times, patientID, etc

Justification: Using a conflict check (e.g. checking if staff have booked the same room with the same time frame), prevents staff from double booking the same hospital rooms at the same time. This reduces the chance of clashes between staff members, over schedule times.

8) Staff availability

INPUT	PROCESS	STORAGE	OUTPUT
StaffID, staff (FirstName), Staff (Surname)	Derive busy slots from ROOM_BOOKINGS	Insert into STAFF_AVAILABILITY table/row (BusyFromTime, ToTime, Status)	Free/busy timeline (e.g. times when staff are busy or not -> BookingID/Booking times), Message “(StaffID) is available/busy”

Justification: Deriving staff availability from bookings ensures real time accuracy without the need for manual updates. This will reduce staff burnout by making staff status and duties/workload visible to others.

9) Login & Authorise New Staff (Security)

INPUT	PROCESS	STORAGE	OUTPUT
Username, Password	Hash password, compare to USER_ACCOUNTS, on "sign in new staff", verify credentials of existing staff then create account.	Insert into USER_ACCOUNTS table/row, store in hash table	Login success or error. For sign-up, message "Account created" or "login successful, welcome (StaffID)"

Justification: By making new users enter the system, using the credentials of another staff member, allows hospitals and staff to protect staff and patient data securely. Linking accounts to staff ID provides accountability, whilst the examination log allows tracking of all system access.

10) Medication Stock: Low-Stock & Resupply

INPUT	PROCESS	STORAGE	OUTPUT
MedicationID, CurrentQuantity, ReorderLevel	Checks if CurrentQuantity < ReorderLevel. If so, flag as "low on stock". on resupply, reset CurrentQuantity to max stock and add timestamp.	Update and insert into MEDICATION_STOCK table/row	Green bar/number updated. "Low stock" pop-up cleared

Justification: Monitoring stock levels automatically, allows hospitals to reduce the chance of running out of critical medication. The reorder system helps staff to restock on medication passively, which supports efficient resource management.

3.2.2 b) Using Pseudocode, explain the different parts of the solution and how these parts will form a complete solution.

My parts of the computer system will be represented using pseudocode, instead of flowcharts, as it's easier to create the main code for the system, compared to flowcharts. I have translated the different sub-programs from my decomposition diagram into different pseudocode programs:

Pseudocode 1 (Medication stock levels):

```
record Medication
   id
   name
```

```
    category
    description
    maxStock
    reorderLevel
    currentQty
endrecord

dictionary meds

procedure saveMedication(m)
endprocedure

function loadMedication(id)
endfunction

function loadAllMedications()
endfunction

function generateId()
endfunction

function isPositiveInteger(x)
    return typeOf(x) = integer AND x > 0
endfunction

function keepWithinRange(value, low, high)
    if value < low then
        return low
    elseif value > high then
        return high
    else
        return value
    endif
endfunction

function addMedication(name, category, description, maxStock, reorderLevel)
    if name = "" OR category = "" then
        return "ERROR: name and category required"
    endif
    if NOT isPositiveInteger(maxStock) then
        return "ERROR: maxStock must be a positive integer"
    endif
```

```
    if NOT isPositiveInteger(reorderLevel) OR reorderLevel >= maxStock then
        return "ERROR: reorderLevel must be a positive integer less than maxStock"
    endif

    m = new Medication
    m.id = generateId()
    m.name = name
    m.category = category
    m.description = description
    m.maxStock = maxStock
    m.reorderLevel = reorderLevel
    m.currentQty = maxStock

    meds[m.id] = m
    saveMedication(m)
    return m.id
endfunction

Function initialiseAmount(m.currentQty, maxStock)
    Amount = maxStock - m.currentQty
    Return Amount
Endfunction

function resupplyMedication(id, amount)
    if NOT isPositiveInteger(amount) then
        return "ERROR: amount must be a positive integer"
    endif

    m = loadMedication(id)
    if m = null then
        return "ERROR: medication not found"
    endif

    m.currentQty = keepWithinRange(m.currentQty + amount, 0, m.maxStock)
    meds[id] = m
    saveMedication(m)

    if m.currentQty <= m.reorderLevel then
        return "OK: resupplied but still low"
    else
        return "OK: resupplied"
    endif
```

```
endfunction

function consumeMedication(id, amount)
   if NOT isPositiveInteger(amount) then
      return "ERROR: amount must be a positive integer"
   endif

   m = loadMedication(id)
   if m = null then
      return "ERROR: medication not found"
   endif

   if amount > m.currentQty then
      return "ERROR: insufficient stock"
   endif

   m.currentQty = m.currentQty - amount
   meds[id] = m
   saveMedication(m)

   if m.currentQty <= m.reorderLevel then
      return "OK: consumed, now low"
   else
      return "OK: consumed"
   endif
endfunction

function getMedication(id)
   return loadMedication(id)
endfunction

function listMedicationsByName()
   list = loadAllMedications()
   sort list by name ascending
   return list
endfunction

function listLowStock()
   all = loadAllMedications()
   lows = new list
   for each m in all do
      if m.currentQty <= m.reorderLevel then
```

```
        append lows, m
      endif
   endfor
   sort lows by (m.currentQty * 100 / m.maxStock) ascending
   return lows
endfunction

function suggestReorderQuantity(id)
   m = loadMedication(id)
   if m = null then
      return "ERROR: medication not found"
   endif
   return m.maxStock - m.currentQty
endfunction
```

Pseudocode 2 (EHR (Electronic Health Records) system):

```
record Patient
   id
   firstName
   lastName
   email
   phone
   dob
   medicalConditions
   medications
   created_at
   updated_at
   notes                (optional)
   medicalImages        (optional)
endrecord

dictionary patients

procedure savePatient(p)
endprocedure

function loadPatient(id)
endfunction

function loadAllPatients()
endfunction
```

```
function generateId()
endfunction

function nowTimestamp()
endfunction

function removeSpace(s)
   return s without leading spaces
endfunction

function isNonEmptyString(s)
   return typeOf(s) = string AND trim(s) <> ""
endfunction

function isValidEmail(s)
   return contains(s,"@") AND contains(s,".")
endfunction

function isDigitsOnly(s)
   i <- 1
   while i <= length(s) do
      if s[i] < "0" OR s[i] > "9" then
         return FALSE
      endif
      i <- i + 1
   endwhile
   return TRUE
endfunction

function isValidPhone(s)
   t <- trim(s)
   return isDigitsOnly(t) AND length(t) >= 7 AND length(t) <= 15
endfunction

function isValidDate(d)
   return typeOf(d) = date
endfunction

function addPatient(firstName, lastName, email, phone, dob, medicalConditions,
medications)
   if NOT isNonEmptyString(firstName) OR NOT isNonEmptyString(lastName) then
```

```
        return "ERROR: name required"
    endif
    if NOT isValidEmail(email) then
        return "ERROR: invalid email"
    endif
    if NOT isValidPhone(phone) then
        return "ERROR: invalid phone"
    endif
    if NOT isValidDate(dob) then
        return "ERROR: invalid dob"
    endif

    p = new Patient
    p.id = generateId()
    p.firstName = trim(firstName)
    p.lastName = trim(lastName)
    p.email = lower(trim(email))
    p.phone = trim(phone)
    p.dob = dob
    p.medicalConditions = medicalConditions
    p.medications = medications
    p.created_at = nowTimestamp()
    p.updated_at = p.created_at

    patients[p.id] = p
    savePatient(p)
    return p.id
endfunction

function viewPatient(id)
    return loadPatient(id)
endfunction

function listPatientsByLastName()
    list = loadAllPatients()
    sort list by lastName ascending, firstName ascending
    return list
endfunction

function searchPatientsByText(q)
    t = lower(trim(q))
    all = loadAllPatients()
```

```
    result = new list
    for each p in all do
        full = lower(p.firstName + " " + p.lastName + " " + p.email + " " + p.phone + " " +
p.medicalConditions + " " + p.medications)
        if contains(full, t) then
            append result, p
        endif
    endfor
    return result
endfunction

function updatePatient(id, firstName, lastName, email, phone, dob,
medicalConditions, medications)
    p = loadPatient(id)
    if p = null then
        return "ERROR: patient not found"
    endif
    if NOT isNonEmptyString(firstName) OR NOT isNonEmptyString(lastName) then
        return "ERROR: name required"
    endif
    if NOT isValidEmail(email) then
        return "ERROR: invalid email"
    endif
    if NOT isValidPhone(phone) then
        return "ERROR: invalid phone"
    endif
    if NOT isValidDate(dob) then
        return "ERROR: invalid dob"
    endif

    p.firstName = trim(firstName)
    p.lastName = trim(lastName)
    p.email = lower(trim(email))
    p.phone = trim(phone)
    p.dob = dob
    p.medicalConditions = medicalConditions
    p.medications = medications
    p.updated_at = nowTimestamp()

    patients[id] = p
    savePatient(p)
    return "OK: updated"
```

```
endfunction

function deletePatient(id)
   p = loadPatient(id)
   if p = null then
      return "ERROR: patient not found"
   endif
   remove patients[id]
   savePatient(p)
   return "OK: deleted"
endfunction
```

Pseudocode 3 (Surgery Room Booking Management)

```
record Room
   id
   label
endrecord

record Booking
   id
   roomId
   staffId
   patientId
   startAt
   endAt
endrecord

dictionary rooms
dictionary bookings

procedure saveRoom(r)
endprocedure

procedure saveBooking(b)
endprocedure

function loadBooking(id)
endfunction

function loadAllBookings()
endfunction
```

```
function generateId()
endfunction

function nowTimestamp()
endfunction

function isValidDateTime(t)
   return typeOf(t) = time
endfunction

function durationMinutes(startAt, endAt)
   return minutes_between(endAt, startAt)
endfunction

function overlaps(aStart, aEnd, bStart, bEnd)
   return NOT (aEnd <= bStart OR bEnd <= aStart)
endfunction

procedure initDefaultRooms()
   rooms[1]  = new Room; rooms[1].id=1;  rooms[1].label="Screening Room 1";
saveRoom(rooms[1])
   rooms[2]  = new Room; rooms[2].id=2;  rooms[2].label="Screening Room 2";
saveRoom(rooms[2])
   rooms[3]  = new Room; rooms[3].id=3;  rooms[3].label="Surgery Room A";
saveRoom(rooms[3])
   rooms[4]  = new Room; rooms[4].id=4;  rooms[4].label="Surgery Room B";
saveRoom(rooms[4])
   rooms[5]  = new Room; rooms[5].id=5;  rooms[5].label="Recovery Bay 1";
saveRoom(rooms[5])
   rooms[6]  = new Room; rooms[6].id=6;  rooms[6].label="Recovery Bay 2";
saveRoom(rooms[6])
   rooms[7]  = new Room; rooms[7].id=7;  rooms[7].label="Consultation 1";
saveRoom(rooms[7])
   rooms[8]  = new Room; rooms[8].id=8;  rooms[8].label="Consultation 2";
saveRoom(rooms[8])
   rooms[9]  = new Room; rooms[9].id=9;  rooms[9].label="Imaging Suite";
saveRoom(rooms[9])
   rooms[10] = new Room; rooms[10].id=10; rooms[10].label="Procedure Room";
saveRoom(rooms[10])
endprocedure
```

```
function listRooms()
   list = new list
   for each id in rooms do
      append list, rooms[id]
   endfor
   sort list by id ascending
   return list
endfunction

function isRoomAvailable(roomId, startAt, endAt)
   if NOT isValidDateTime(startAt) OR NOT isValidDateTime(endAt) OR endAt <=
startAt then
      return FALSE
   endif
   all = loadAllBookings()
   for each b in all do
      if b.roomId = roomId AND overlaps(b.startAt, b.endAt, startAt, endAt) then
         return FALSE
      endif
   endfor
   return TRUE
endfunction

function listAvailableRooms(startAt, endAt)
   result = new list
   for each r in listRooms() do
      if isRoomAvailable(r.id, startAt, endAt) then
         append result, r
      endif
   endfor
   return result
endfunction

function listBookedRooms(startAt, endAt)
   result = new list
   all = loadAllBookings()
   for each b in all do
      if overlaps(b.startAt, b.endAt, startAt, endAt) then
         append result, b
      endif
   endfor
   sort result by startAt ascending, roomId ascending
```

```
    return result
endfunction

function bookingDuration(b)
    return durationMinutes(b.startAt, b.endAt)
endfunction

function bookRoom(roomId, staffId, patientId, startAt, endAt)
    if rooms[roomId] = null then
        return "ERROR: room not found"
    endif
    if NOT isValidDateTime(startAt) OR NOT isValidDateTime(endAt) OR endAt <=
startAt then
        return "ERROR: invalid time range"
    endif
    mins = durationMinutes(startAt, endAt)
    if mins < 0 OR mins > 16 then
        return "ERROR: duration must be between 1 to 15 minutes"
    endif
    if NOT isRoomAvailable(roomId, startAt, endAt) then
        return "ERROR: room not available"
    endif

    b = new Booking
    b.id = generateId()
    b.roomId = roomId
    b.staffId = staffId
    b.patientId = patientId
    b.startAt = startAt
    b.endAt = endAt
    b.created_at = nowTimestamp()

    bookings[b.id] = b
    saveBooking(b)
    return b.id
endfunction

function viewBooking(id)
    return loadBooking(id)
endfunction

function listBookingsByRoom(roomId)
```

```
    all = loadAllBookings()
    result = new list
    for each b in all do
        if b.roomId = roomId then
            append result, b
        endif
    endfor
    sort result by startAt ascending
    return result
endfunction

function cancelBooking(id)
    b = loadBooking(id)
    if b = null then
        return "ERROR: booking not found"
    endif
    remove bookings[id]
    saveBooking(b)
    return "OK: cancelled"
endfunction
```

Pseudocode 4 (View Staff availability)

```
record Staff
    id
    firstName
    lastName
    role
endrecord

dictionary staff

procedure saveStaff(s)
endprocedure

function loadAllStaff()
endfunction

function loadBooking(id)
endfunction

function loadAllBookings()
```

```
endfunction

procedure saveBooking(b)
endprocedure

function nowTimestamp()
endfunction

function isValidDateTime(t)
   return typeOf(t) = datetime
endfunction

function getActualEnd(b)
   if b.actualEndAt => null then
      return b.actualEndAt
   else
      return b.endAt
   endif
endfunction

function isTimeBetween(startAt, endAt, t)
   return (t >= startAt) AND (t <= endAt)
endfunction

function isStaffBusyAt(staffId, t)
   if NOT isValidDateTime(t) then
      return FALSE
   endif
   all = loadAllBookings()
   for each b in all do
      if b.staffId = staffId then
         eEnd = getEffectiveEnd(b)
         if isInstantWithin(b.startAt, eEnd, t) then
            return TRUE
         endif
      endif
   endfor
   return FALSE
endfunction

function listAvailableStaffAt(t)
   result = new list
```

```
    allStaff = loadAllStaff()
    for each s in allStaff do
        if NOT isStaffBusyAt(s.id, t) then
            append result, s
        endif
    endfor
    sort result by lastName ascending, firstName ascending
    return result
endfunction

function listUnavailableStaffAt(t)
    result = new list
    allStaff = loadAllStaff()
    for each s in allStaff do
        if isStaffBusyAt(s.id, t) then
            append result, s
        endif
    endfor
    sort result by lastName ascending, firstName ascending
    return result
endfunction

function listAvailableStaffNow()
    return listAvailableStaffAt(nowTimestamp())
endfunction

function listUnavailableStaffNow()
    return listUnavailableStaffAt(nowTimestamp())
endfunction

function checkOutBooking(staffId, bookingId, checkoutTime)
    b = loadBooking(bookingId)
    if b = null OR b.staffId => staffId then
        return "ERROR: booking not found"
    endif
    if NOT isValidDateTime(checkoutTime) then
        return "ERROR: invalid time"
    endif
    if checkoutTime <= b.startAt then
        return "ERROR: checkout must be after start"
    endif
    if checkoutTime > b.endAt then
```

```
        checkoutTime = b.endAt
    endif
    b.actualEndAt = checkoutTime
    saveBooking(b)
    return "OK: checked out"
endfunction
```

Pseudocode 5 (View Medical Images)

```
record ImageRef
    id
    patientId
    type                # "XRAY", "CT", "MRI"
    filePath
    takenAt
    uploadedAt
    notes
endrecord

dictionary images

procedure saveImage(i)
endprocedure

function loadImage(id)
endfunction

function loadAllImages()
endfunction

function generateId()
endfunction

function nowTimestamp()
endfunction

function isValidType(m)
    return m = "XRAY" OR m = "CT" OR m = "MRI"
endfunction

function uploadImage(patientId, type, filePath, takenAt, notes)
    if NOT isValidType(modality) then
```

```
        return "ERROR: invalid type"
    endif
    if patientId = "" OR filePath = "" then
        return "ERROR: missing data"
    endif

    i = new ImageRef
    i.id = generateId()
    i.patientId = patientId
    i.type = type
    i.filePath = filePath
    i.takenAt = takenAt
    i.uploadedAt = nowTimestamp()
    i.notes = notes

    images[i.id] = i
    saveImage(i)
    return i.id
endfunction

function listImagesByPatient(patientId)
    result = new list
    all = loadAllImages()
    for each i in all do
        if i.patientId = patientId then
            append result, i
        endif
    endfor
    sort result by uploadedAt descending
    return result
endfunction

function listImagesByType(patientId, type)
    result = new list
    all = loadAllImages()
    for each i in all do
        if i.patientId = patientId AND i.type = type then
            append result, i
        endif
    endfor
    sort result by uploadedAt descending
    return result
```

```
endfunction

function getLatestImages(patientId, limit)
   all = listImagesByPatient(patientId)
   result = firstN(all, limit)
   return result
endfunction

function viewImage(id)
   i = loadImage(id)
   if i = null then
      return "ERROR: image not found"
   endif
   openImage(i.filePath)
   return "OK: image opened"
endfunction
```

Pseudocode 6 (Automated medication reminders)

```
record MedReminder
   id
   patientId
   medicationId
   doseAmount
   scheduleType          # "TIME_OF_DAY" or "EVERY_X_HOURS"
   scheduleData          # e.g. "08:00,20:00" or "5" (hours)
   nextDueAt
   active
   created_at
endrecord

dictionary reminders

procedure saveReminder(r)
endprocedure

function loadReminder(id)
endfunction

function loadAllReminders()
endfunction
```

```
function generateId()
endfunction

function nowTimestamp()
endfunction

function isValidDateTime(t)
   return typeOf(t) = datetime
endfunction

function isPositiveInteger(x)
   return typeOf(x) = integer AND x > 0
endfunction

function isValidTimeText(s)                    # "HH:MM"
   return length(s)=5 AND s[3]=":" AND
         isPositiveInteger(toInteger(s[1..2])) AND
         isPositiveInteger(toInteger(s[4..5])) AND
         toInteger(s[1..2]) >= 0 AND toInteger(s[1..2]) < 24 AND
         toInteger(s[4..5]) >= 0 AND toInteger(s[4..5]) < 60
endfunction

function addMinutes(dt, mins)
   return dt + mins minutes
endfunction

function minutesBetween(later, earlier)
   return minutes_between(later, earlier)
endfunction

function nextFromTimeList(now, csvTimes)     # csvTimes: "08:00,13:30,20:00"
   times = split(csvTimes, ",")
   todayCandidates = new list
   for each t in times do
      if isValidTimeText(t) then
         h = toInteger(t[1..2]); m = toInteger(t[4..5])
         cand = datetime(dateOf(now), h, m, 0)
         if cand >= now then
            append todayCandidates, cand
         endif
      endif
   endfor
```

```
    if length(todayCandidates) > 0 then
        sort todayCandidates by value ascending
        return todayCandidates[1]
    else
        firstTomorrow = split(csvTimes, ",")[1]
        hh = toInteger(firstTomorrow[1..2]); mm = toInteger(firstTomorrow[4..5])
        return datetime(dateOf(now)+1 day, hh, mm, 0)
    endif
endfunction

function computeNextDue(r, now)
    if r.scheduleType = "TIME_OF_DAY" then
        return nextFromTimeList(addMinutes(now,1), r.scheduleData)
    else
        hrs = toInteger(r.scheduleData)
        return addMinutes(r.nextDueAt, hrs * 60)
    endif
endfunction

function createReminderTimeOfDay(patientId, medicationId, doseAmount, timesCsv)
    if NOT isPositiveInteger(doseAmount) then
        return "ERROR: invalid dose"
    endif
    r = new MedReminder
    r.id = generateId()
    r.patientId = patientId
    r.medicationId = medicationId
    r.doseAmount = doseAmount
    r.scheduleType = "TIME_OF_DAY"
    r.scheduleData = timesCsv
    r.nextDueAt = nextFromTimeList(nowTimestamp(), timesCsv)
    r.active = true
    r.created_at = nowTimestamp()
    reminders[r.id] = r
    saveReminder(r)
    return r.id
endfunction

function createReminderEveryXHours(patientId, medicationId, doseAmount,
everyHours, startAt)
    if NOT isPositiveInteger(doseAmount) OR NOT isPositiveInteger(everyHours) then
        return "ERROR: invalid dose or hours"
```

```
    endif
    if NOT isValidDateTime(startAt) then
        return "ERROR: invalid start time"
    endif
    r = new MedReminder
    r.id = generateId()
    r.patientId = patientId
    r.medicationId = medicationId
    r.doseAmount = doseAmount
    r.scheduleType = "EVERY_X_HOURS"
    r.scheduleData = toString(everyHours)
    if startAt > nowTimestamp() then
        r.nextDueAt = startAt
    else
        // roll forward in steps of everyHours until > now
        stepMins = everyHours * 60
        k = 0
        cand = startAt
        while cand <= nowTimestamp() do
            cand = addMinutes(startAt, (k+1) * stepMins)
            k = k + 1
        endwhile
        r.nextDueAt = cand
    endif
    r.active = true
    r.created_at = nowTimestamp()
    reminders[r.id] = r
    saveReminder(r)
    return r.id
endfunction

function listActiveReminders()
    result = new list
    all = loadAllReminders()
    for each r in all do
        if r.active = true then
            append result, r
        endif
    endfor
    sort result by nextDueAt ascending
    return result
endfunction
```

```
function listRemindersForPatient(patientId)
   result = new list
   all = loadAllReminders()
   for each r in all do
      if r.patientId = patientId AND r.active = true then
         append result, r
      endif
   endfor
   sort result by nextDueAt ascending
   return result
endfunction

function listRemindersDueAt(now)
   due = new list
   all = loadAllReminders()
   for each r in all do
      if r.active = true AND r.nextDueAt <= now then
         append due, r
      endif
   endfor
   return due
endfunction

procedure reminderTick(now)
   dueList = listRemindersDueAt(now)
   for each r in dueList do
      showPopup(r.patientId, r.medicationId, r.doseAmount, r.nextDueAt)
      playSound()
      if userClickedAcknowledge(r.id) then
         msg = consumeMedication(r.medicationId, r.doseAmount)
         a = new AdministrationLog
         a.id = generateId()
         a.reminderId = r.id
         a.patientId = r.patientId
         a.medicationId = r.medicationId
         a.doseAmount = r.doseAmount
         a.administeredAt = now
         a.staffId = currentStaffId()
         a.note = msg
         adminLogs[a.id] = a
         saveAdminLog(a)
```

```
        r.nextDueAt = computeNextDue(r, now)
        saveReminder(r)
      elseif userClickedSnooze(r.id) then
        r.nextDueAt = addMinutes(now, 5)
        saveReminder(r)
      endif
    endfor
endprocedure

function editReminder(id, doseAmount, scheduleType, scheduleData)
    r = loadReminder(id)
    if r = null then
      return "ERROR: reminder not found"
    endif
    if NOT isPositiveInteger(doseAmount) then
      return "ERROR: invalid dose"
    endif
    if scheduleType <> "TIME_OF_DAY" AND scheduleType <> "EVERY_X_HOURS" then
      return "ERROR: invalid schedule"
    endif
    r.doseAmount = doseAmount
    r.scheduleType = scheduleType
    r.scheduleData = scheduleData
    r.nextDueAt = computeNextDue(r, nowTimestamp())
    reminders[id] = r
    saveReminder(r)
    return "OK: updated"
endfunction

function deactivateReminder(id)
    r = loadReminder(id)
    if r = null then
      return "ERROR: reminder not found"
    endif
    r.active = false
    saveReminder(r)
    return "OK: deactivated"
endfunction
```

Pseudocode 7 (Account system & log-in)

```
record User
```

```
    username
    salt
    passwordHash
    created_at
    lastLoginAt
    active
endrecord

record Session
    token
    username
    created_at
    expires_at
    active
endrecord

dictionary users
dictionary sessions

procedure saveUser(u)
endprocedure

function loadUser(username)
endfunction

procedure saveSession(s)
endprocedure

function loadSession(token)
endfunction

function generateId()
endfunction

function nowTimestamp()
endfunction

function generateSalt()
endfunction

function hashPassword(password, salt)
    return HASH(salt + password)
```

```
endfunction

function createUser(username, password)
   if username = "" OR password = "" then
      return "ERROR: missing fields"
   endif
   if users[username] => null then
      return "ERROR: username taken"
   endif
   u = new User
   u.username = username
   u.salt = generateSalt()
   u.passwordHash = hashPassword(password, u.salt)
   u.created_at = nowTimestamp()
   u.lastLoginAt = null
   u.active = true
   users[username] = u
   saveUser(u)
   return "OK: user created"
endfunction

function verifyPassword(username, password)
   u = loadUser(username)
   if u = null OR u.active = false then
      return FALSE
   endif
   return hashPassword(password, u.salt) = u.passwordHash
endfunction

function login(username, password)
   if verifyPassword(username, password) = FALSE then
      return "ERROR: invalid credentials"
   endif
   s = new Session
   s.token = generateId()
   s.username = username
   s.created_at = nowTimestamp()
   s.expires_at = nowTimestamp() + 8 hours
   s.active = true
   sessions[s.token] = s
   saveSession(s)
   u = loadUser(username)
```

```
    u.lastLoginAt = nowTimestamp()
    saveUser(u)
    return s.token
endfunction

function logout(sessionToken)
    s = loadSession(sessionToken)
    if s = null OR s.active = false then
        return "ERROR: session not found"
    endif
    s.active = false
    saveSession(s)
    return "OK: logged out"
endfunction

function loginWithExistingStaffAuthorization(existingUsername, existingPassword,
newUsername, newPassword)
    if verifyPassword(existingUsername, existingPassword) = FALSE then
        return "ERROR: authorising staff credentials invalid"
    endif
    msg = createUser(newUsername, newPassword)
    if left(msg) = "OK:" then
        return login(newUsername, newPassword)
    else
        return msg
    endif
endfunction

function changePassword(username, oldPassword, newPassword)
    if verifyPassword(username, oldPassword) = FALSE then
        return "ERROR: invalid current password"
    endif
    u = loadUser(username)
    u.salt = generateSalt()
    u.passwordHash = hashPassword(newPassword, u.salt)
    saveUser(u)
    return "OK: password changed"
endfunction

function isSessionValid(sessionToken)
    s = loadSession(sessionToken)
    if s = null then
```

```
    return FALSE
  endif
  if s.active = false then
    return FALSE
  endif
  if nowTimestamp() > s.expires_at then
    s.active = false
    saveSession(s)
    return FALSE
  endif
  return TRUE
Endfunction
```

3.2.2c) What are the usability features of the solution?

Usability features are the characteristics of a product that determine how easy, effective, and interactive it is to use, in this case it's the hospital platform. It is important that this system is able to provide an interactive and easy-to-use interface to hospital staff and that this GUI design caters to the "4 golden rules of UI design". Those rules being:

1. **Consistency -** This ensures that the behaviour of the GUI stays consistent throughout the interface. This includes using fixed colours, fonts, and layouts, to help hospital staff become familiar with the interface quickly. Through, this rule, I will also implement one my accessibility requirements which is having a clean and easy-to-use interface with high contrast colours (e.g., dark text on white background) and make that the consistent colour scheme

2. **User control and freedom** - This rule emphasises how users (hospital staff) should feel in control of their interactions with the interface and should have the ability to easily navigate the GUI (e.g., having "undo" actions) and clear navigation paths the user can take.

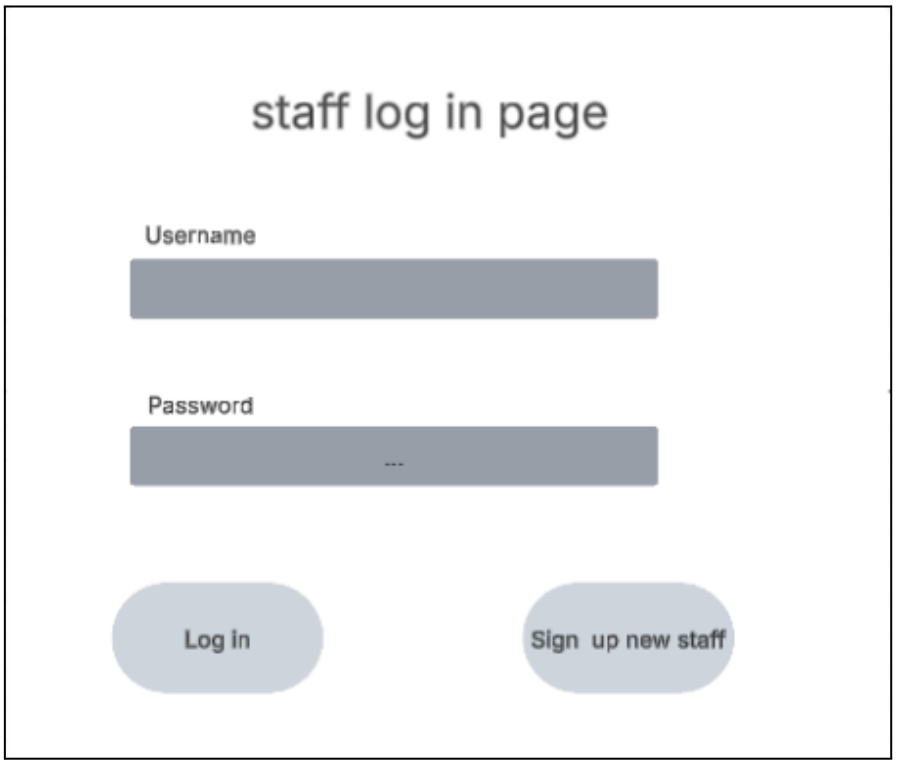
3. **Error prevention** - This focuses on designing systems that minimise chances of input errors happening in the first place. This can be done through input validation and clear instructions passed to the user. This is also another accessibility requirement is to provide "clear, user-friendly error messages" to hospital staff (e.g., Patient Phone (phone number) must be longer than 8 characters). This creates a smoother experience for all hospital staff.

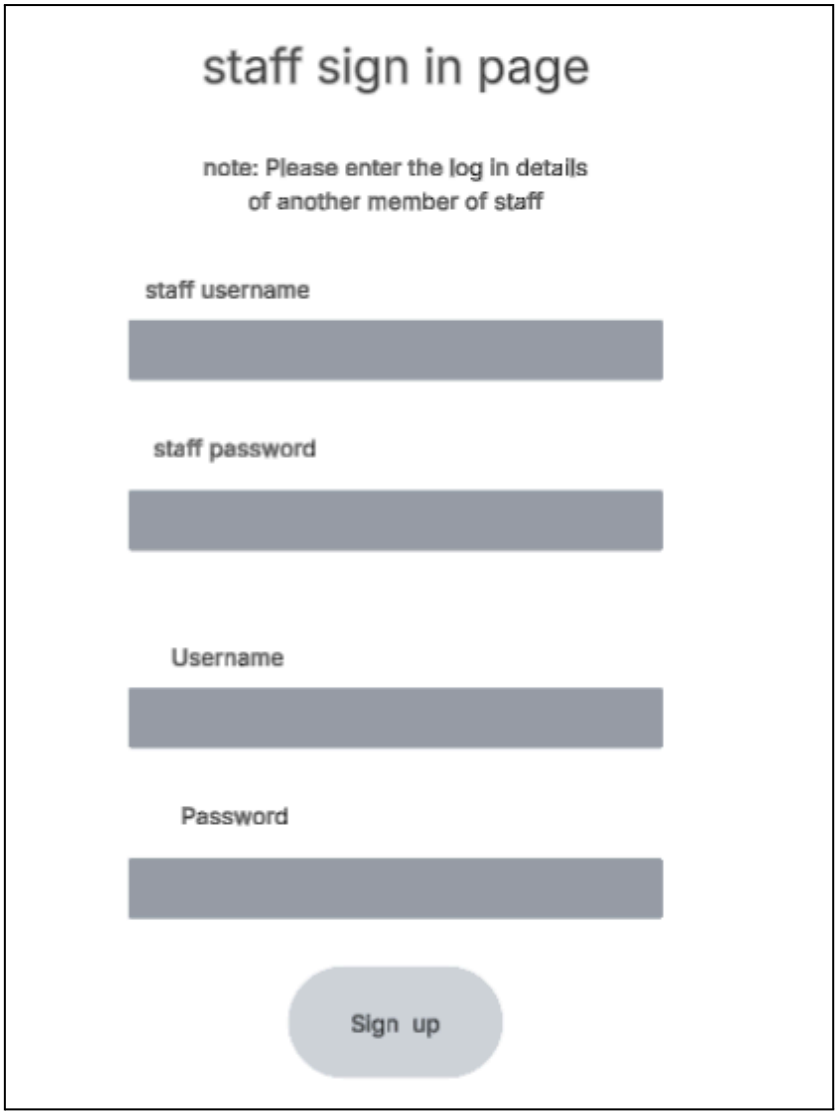
4. **Feedback** - This rule is crucial, as it informs users about the results of the different actions they do (e.g., resupplying medicine, updating patient records, etc). This will also include visual (pop-up notifications) and vocal (sound alerts

for medicine). This reassures hospital staff that their input was recognised and that the action was successful.

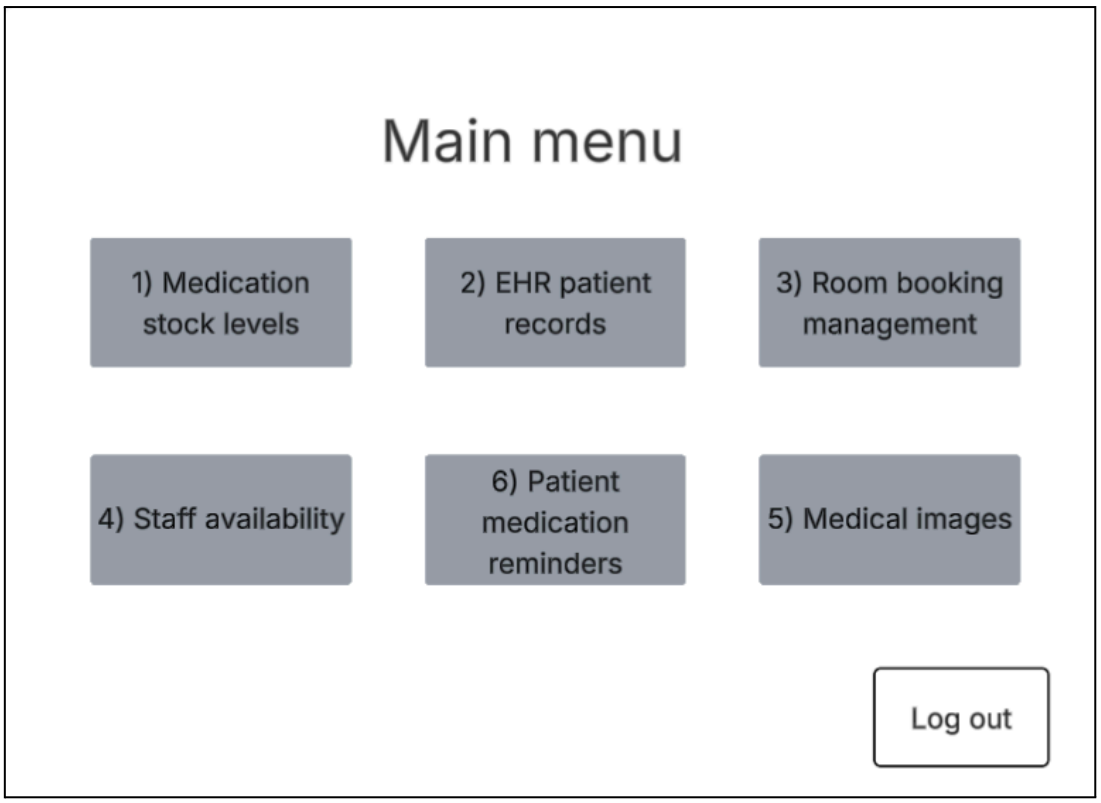
Below, I have designed some wireframes for the GUI of the hospital system, and how it will be designed to follow these golden rules of UI design:

1) **Login pages:**
   These wireframes simply show how the staff login pages will work, where the users will be able to enter their username and password into the spaces provided. Then they can sign up using the button and log in using another button. Additionally, if users click on the sign up new staff button, it will bring them to a new page asking them to enter their own credentials, as well as the credentials of an existing staff member.



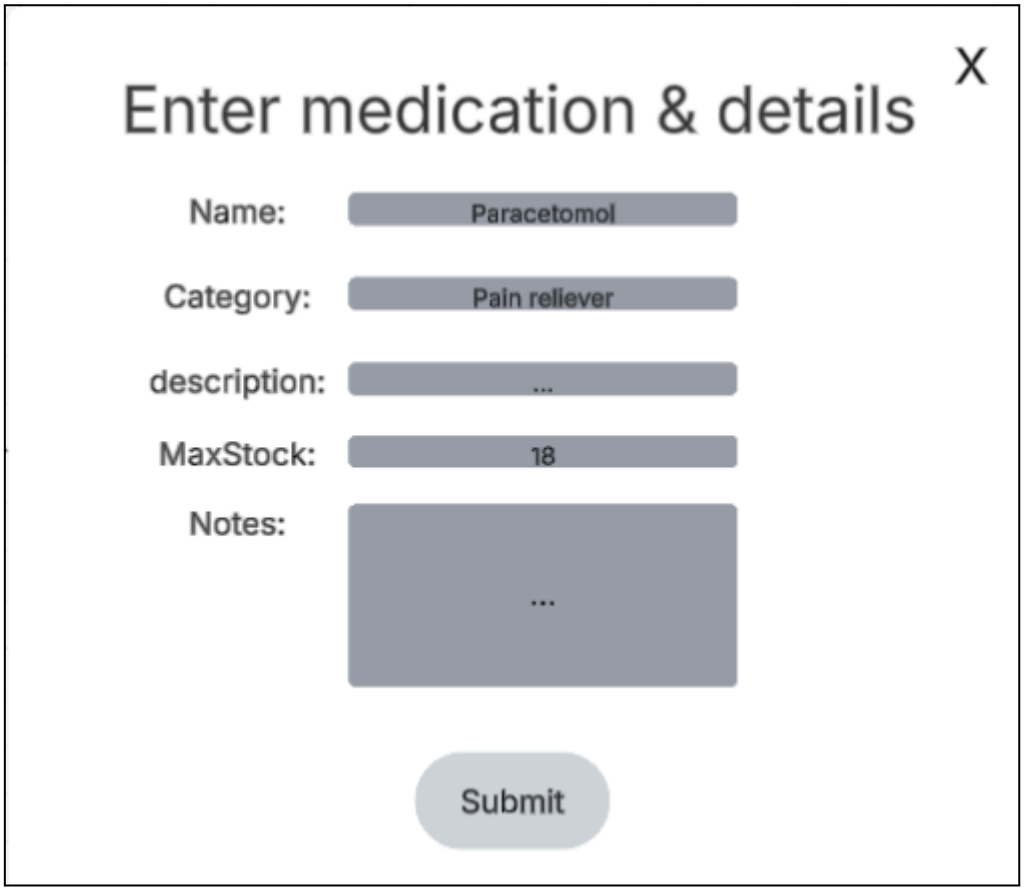


2) **Main Menu:**
This will showcase what you will see directly after you log in to the system. This allows hospital staff to see all the system has to offer in one place. Additionally, on some wireframes, I have included a hamburger menu icon so that users can either go back to the main menu or access another part of the system, without having to access the main menu.

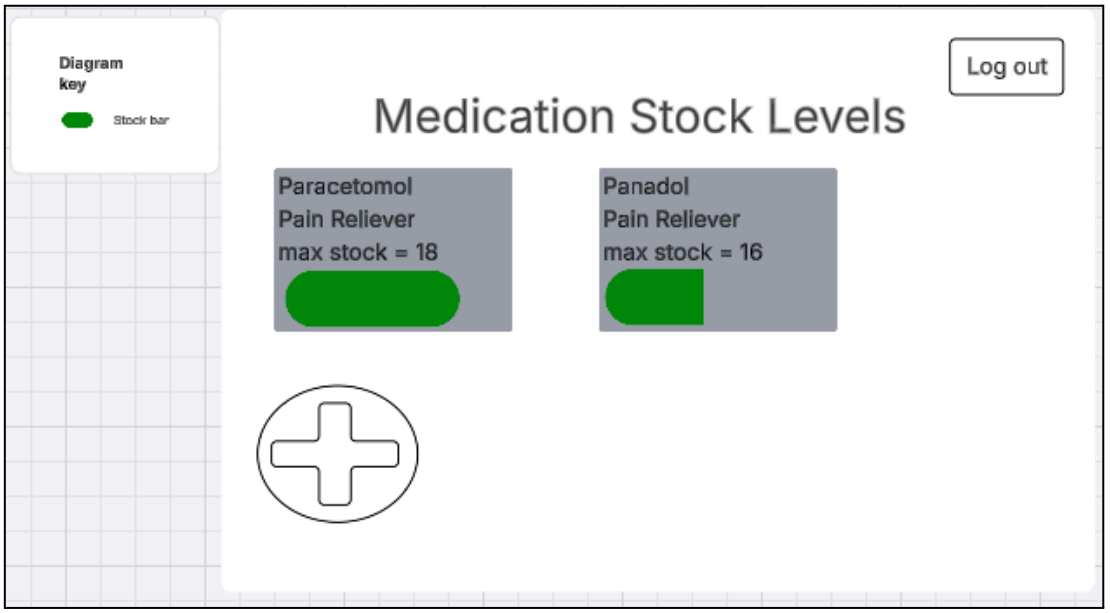


3) **Medication stock levels**
When new members of staff log in, the medication stock levels will not show anything because the user still hasn't put any medication on the system(1). However if a user does enter a medication(2), This will be displayed in the main overview/summary page of the medication stock levels function(3). From this overview, users can select any medication to view its details and quantity information(4), or resupply if low(5-pop-up)

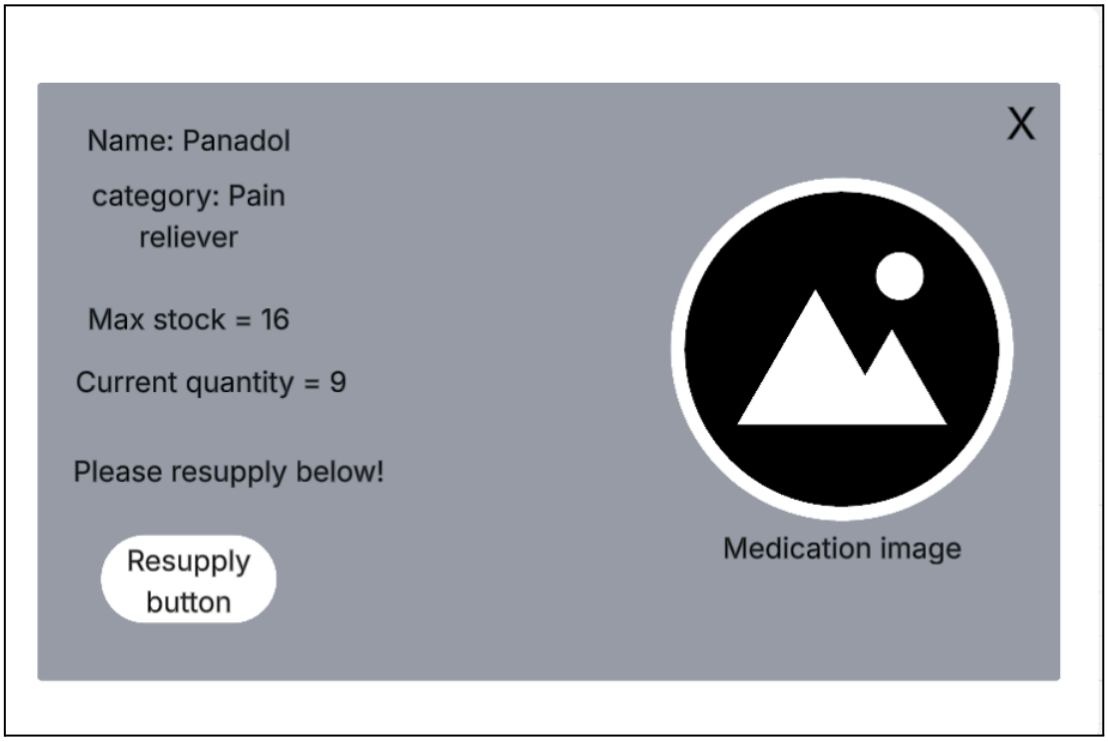
This will be what users see as soon as they click the first option. It will tell them that they don't have any registered medication and that they can click on the plus sign to add them to the system.



When they click on the plus sign, this is what they will see. It will ask them to fill in all of the information in these fields. You will also notice that I added a close button so that users can click off if they don't want to enter medication, on the system.



This part shows them the overall stock levels of all registered medication on the system, where the green bar represents their stock. If the stock is at maximum, (e.g., by resupplying or initialising the max stock when entering the medication) this will showcase a full green bar, indicating that the stock level for that medication is at full capacity. However, if the medication is not at full capacity (i.e. current quantity is lower than max stock), the green bar will be displayed as shorter, thus indicating that there is lower stock available.



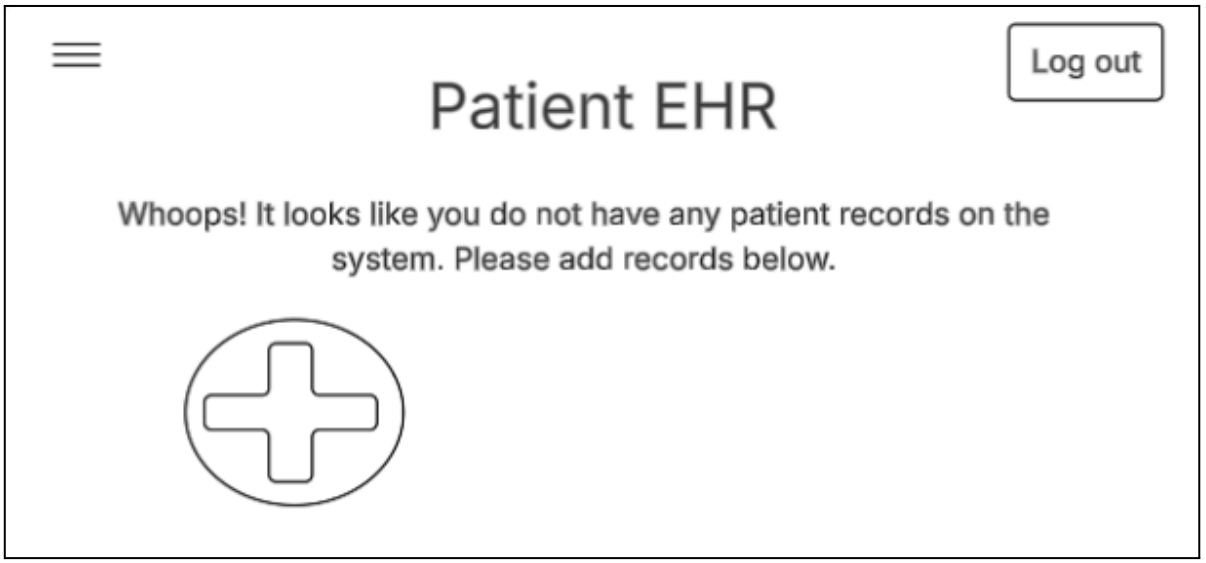
When you click on a specific medication on the overview screen of the stock levels page, it will bring you to the following screen, where you can view medication information, and resupply if medication is low.

The following wireframe is meant to be a pop-up notification, occurring when the user is doing something else on the system (e.g. updating EHR). This will likely show up on the top of the screen in which the user can also click off the alert by pressing the following buttons.

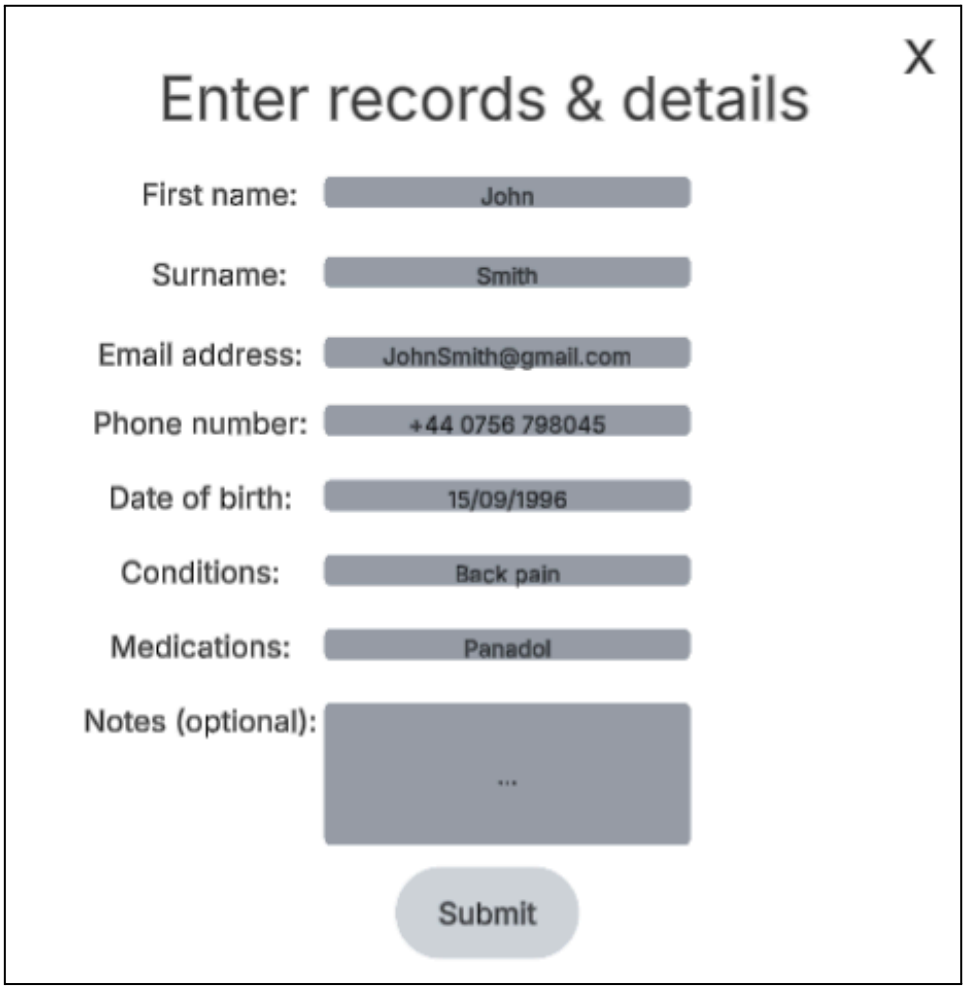
4) EHR (Electronic Health Records) system

Like the medication stock levels function, the EHR system will not show any patient records when new staff sign up to the system, there will not be any records present and staff will need to enter them(1). They are then able to go to a page where they can enter patient details and information(2).

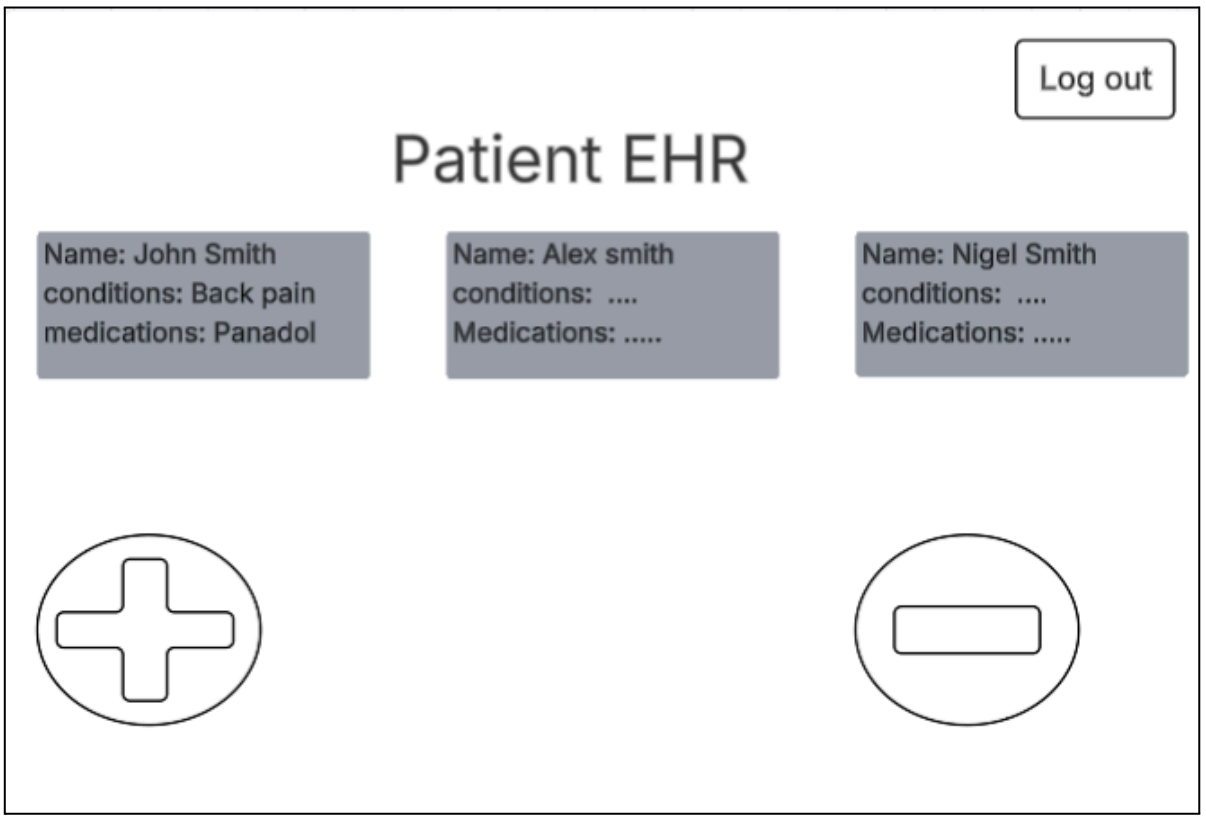
They are also able to view the records as a summarised view, just like the medication stock levels, the difference is that there will be 2 symbols i.e one for adding records (plus symbol) and one for deleting them (minus symbol)(3). They can also click on any records to view the information or they can update them by clicking another button(4).



This shows staff that there are no EHRs registered on the system, and functions just like the stock level wireframe. Again they can click on the plus symbol to add patients onto the system.



When staff click on the plus symbol, it takes them to this page where they can enter these details. You may notice that I have put an exit symbol, to allow staff/users to exit this page, if they wish.



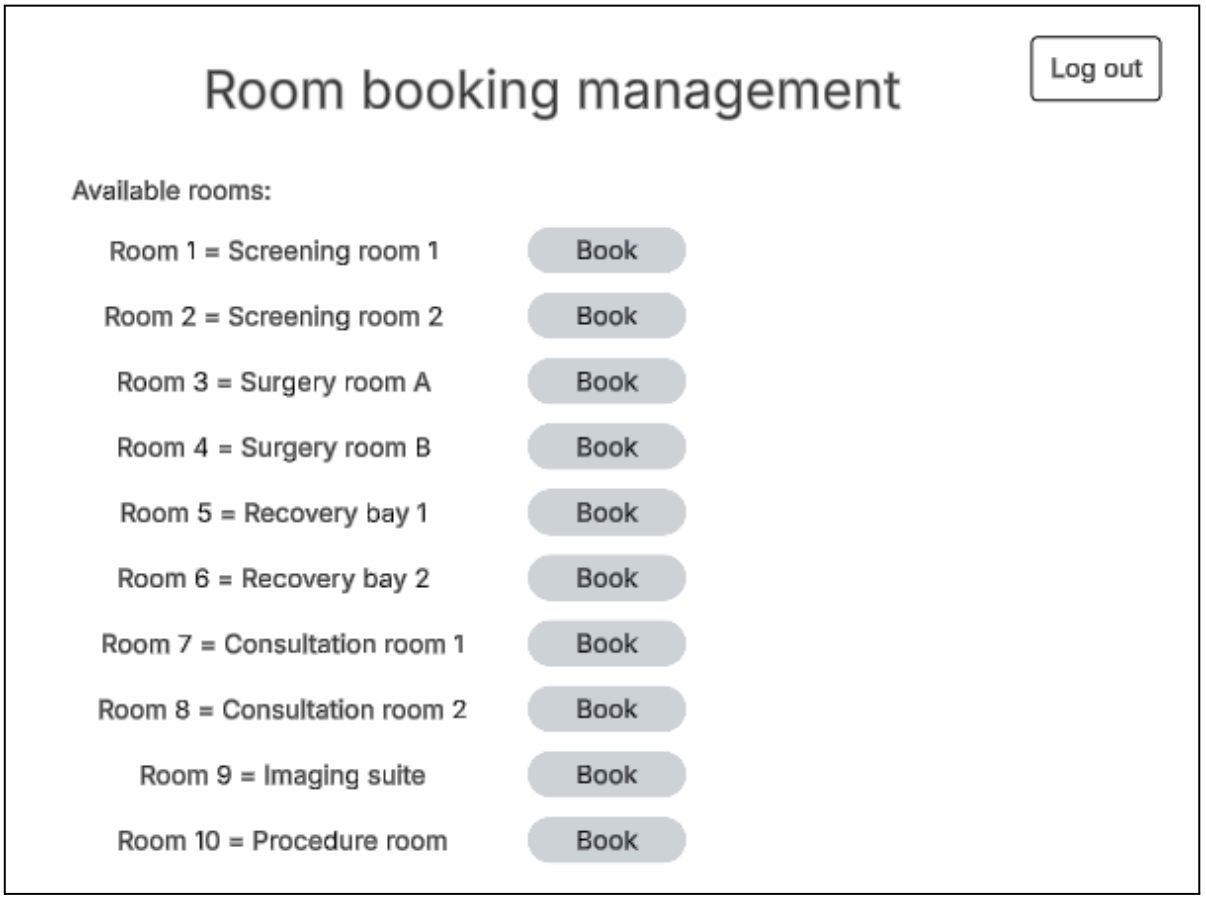
Now when they submit these records they will be able to view them like a summary page, again just like the medication overview page. I have also added a subtract symbol, if users want to delete records. I have added a few more instances (Alex Smith, and Nigel Smith) for references.

If users click on specific records from the last page/mockup, they can view the records in more detail or update them. If they click on the update button, it will bring them back to the page where they can enter the records and details again, with the information about the patient saved.

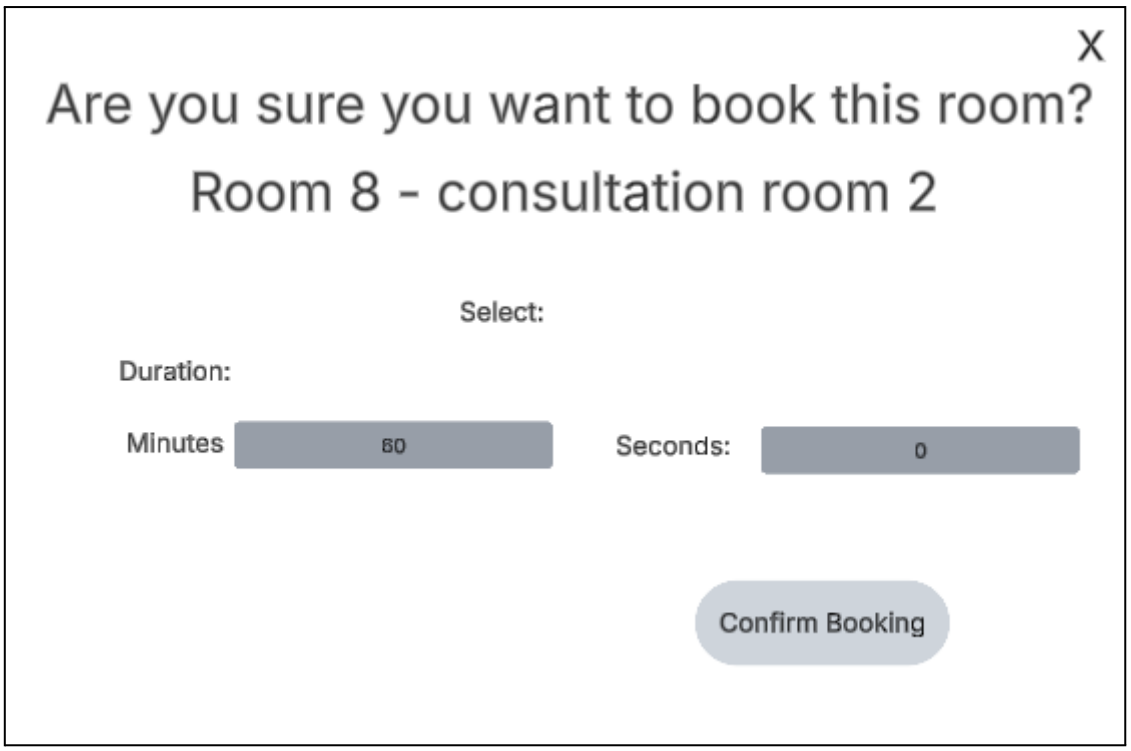
5) **Surgery room booking management**

The wireframes for this function will be different. At the start, when 0 staff are registered, this will mean that all rooms are available for booking (1). However, when staff are registered on the system (i.e. using the details of another staff member & going through the process of hashing), they will be able to book rooms separately. And, when that happens, the room they booked will be displayed as unavailable(2). Additionally, users can also book the rooms themselves and choose how long they are booked for, with a maximum time of 5 hours (actually just 10 minutes) (3). The rooms will be displayed in this order:

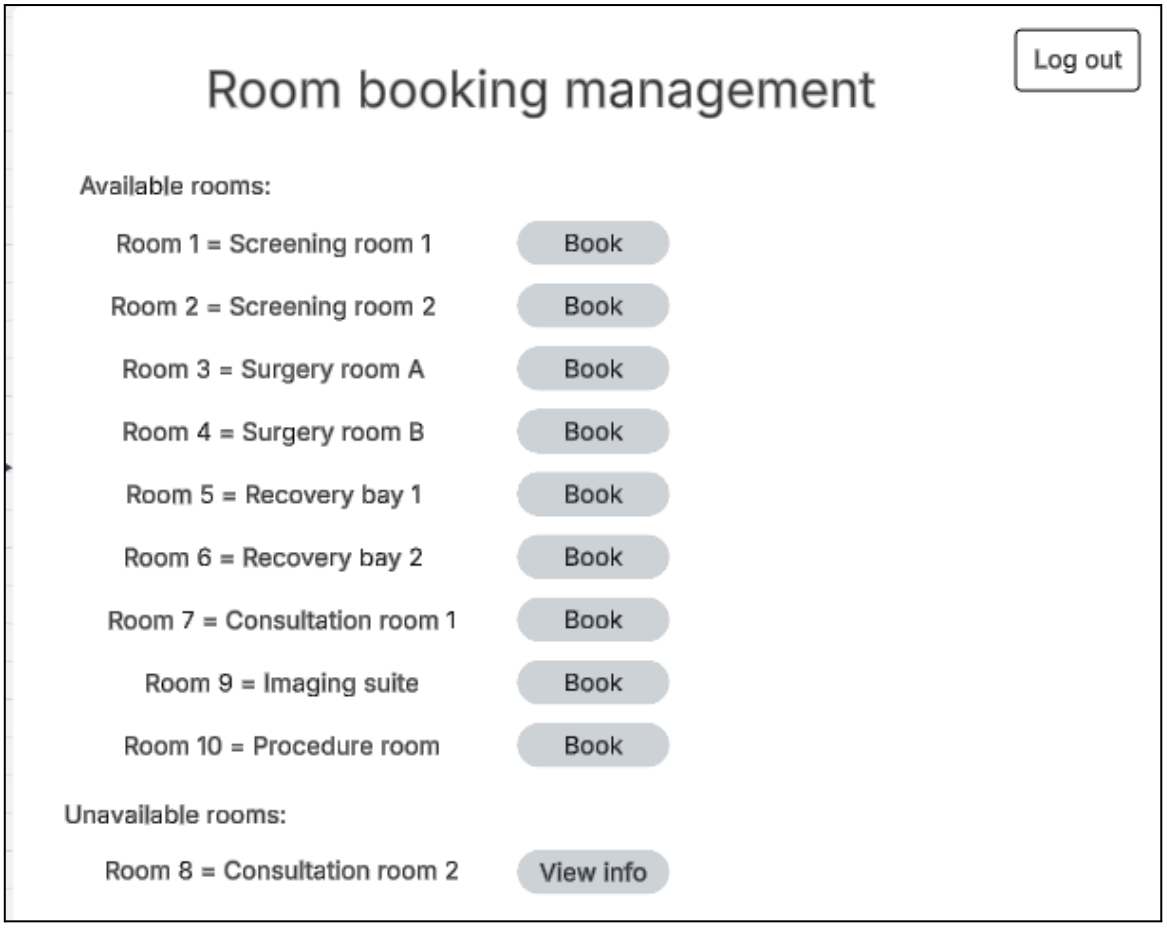
- Room 1 = “Screening room 1”
- Room 2 = “Screening room 2”
- Room 3 = “Surgery room A”
- Room 4 = “Surgery room B”
- Room 5 = “Recovery bay 1”
- Room 6 = “Recovery bay 2”
- Room 7 = “Consultation room 1”
- Room 8 = ”Consultation room 2”
- Room 9 = “Imaging suite”
- Room 10 = “Procedure room”



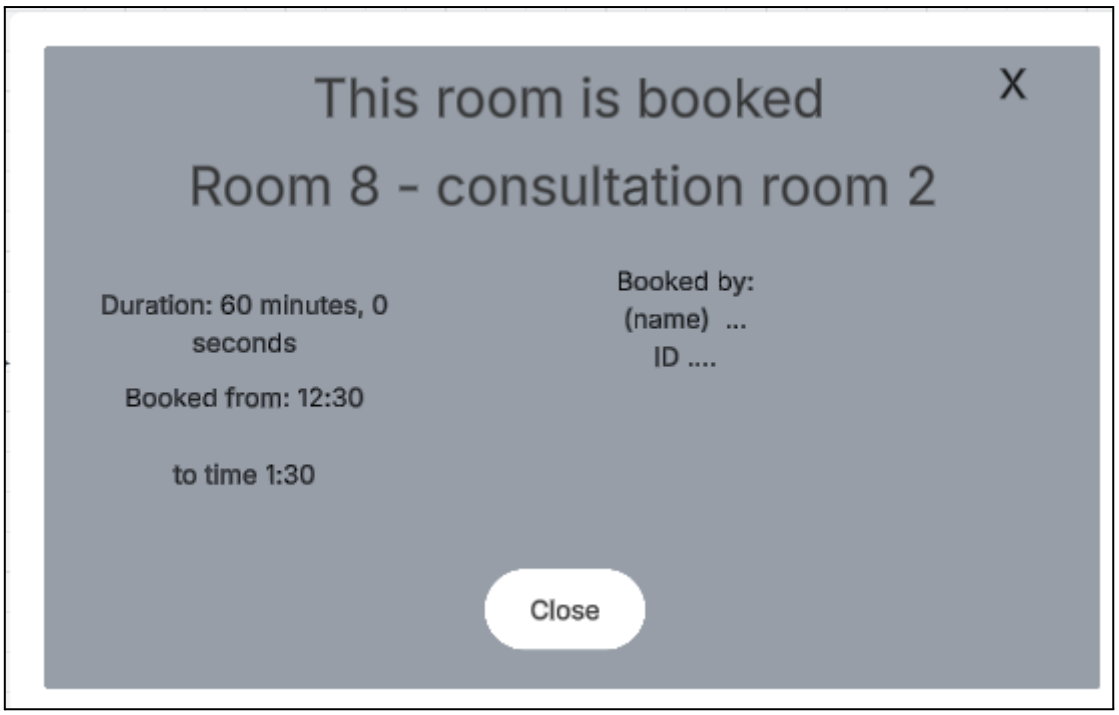
This screen is what new staff will see as soon as they sign up to the system. At first, it will display that all rooms are available for booking by users. But when multiple staff sign up on the same computer, they are able to see if rooms are booked and whom it was booked by.



If you click on one of the buttons for booking, it will bring you to this page which will allow you to book it and choose how long it's booked. In this case, I chose room 8 (consultation room 2) for briefing, & I am booking it for 60 minutes. Then click confirm booking



Back to this screen, as you can see, I have successfully booked the consultation room 2. From this point, this will allow users to book other available rooms or view information of rooms that are booked. This will also be what new users will see if they use the same desktop computer, to access the system, only if the previous user has booked a room. Right now, this is the same account.



If users click on the view info button, they are able to see how long each room has been booked for and who booked it, by displaying their name and user ID. From this point, I have added two ways where users can close the pop-up box; either by using the x button in the top-right corner or using the close tab at the bottom.

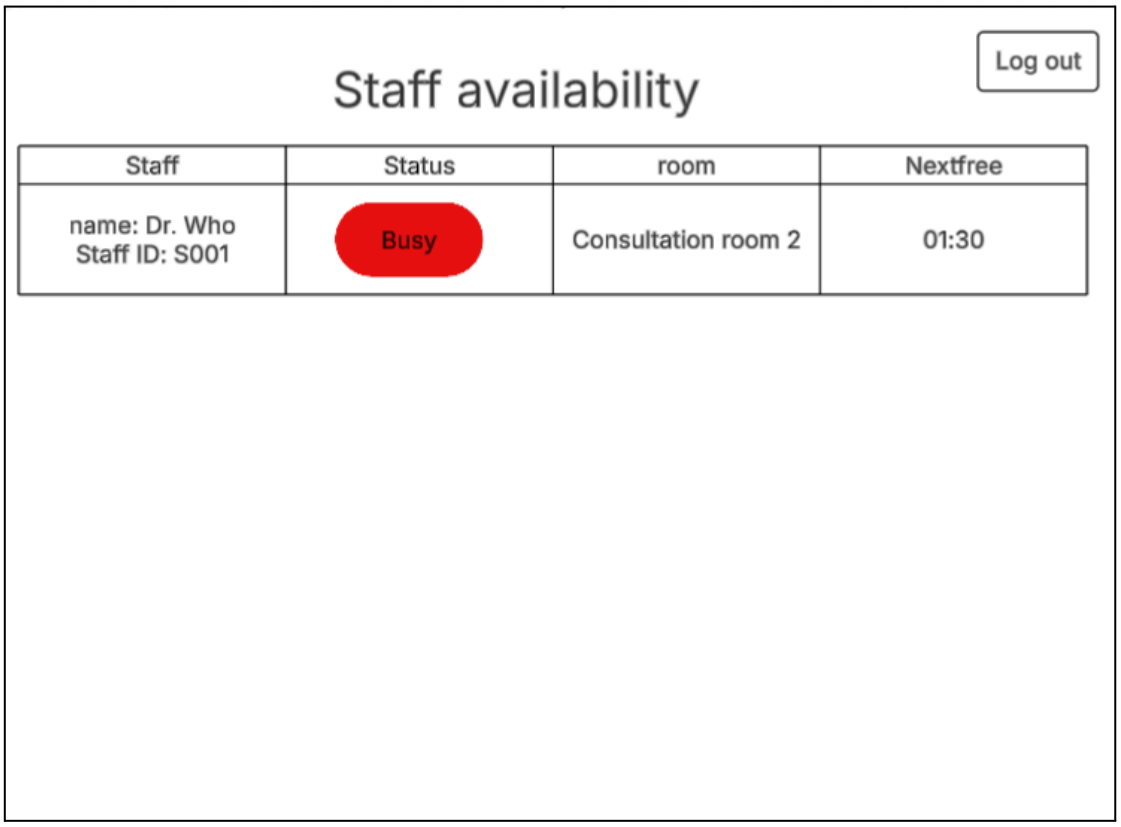
6) **Staff availability module**

Unlike the wireframes above, new staff are able to see all previous staff members who used the system, staff information, and whether they are available or not. This is as long as they used the same desktop computer to access the system. You may have noticed that in the previous wireframes that I added, there is a button in the top right corner that allows staff to log out of their account to let other staff use the system, when they're finished. This allows multiple users to login to the system, without having multiple computers. In this case let's make a random account

Lets imagine that this was the user that was conducting all of the actions in the previous page e.g. adding medications, adding the patient EHRs and booking room 8 (consultation room 2). Now imagine, Dr. who logged out. Now let's create another account.... This time, it's another user

Now that Dr. Why has logged in, he is able to access all features of the system, including the view staff availability feature, which I will get into in more detail. Now, then the wireframe will be presented in table form to show who is available or not. If

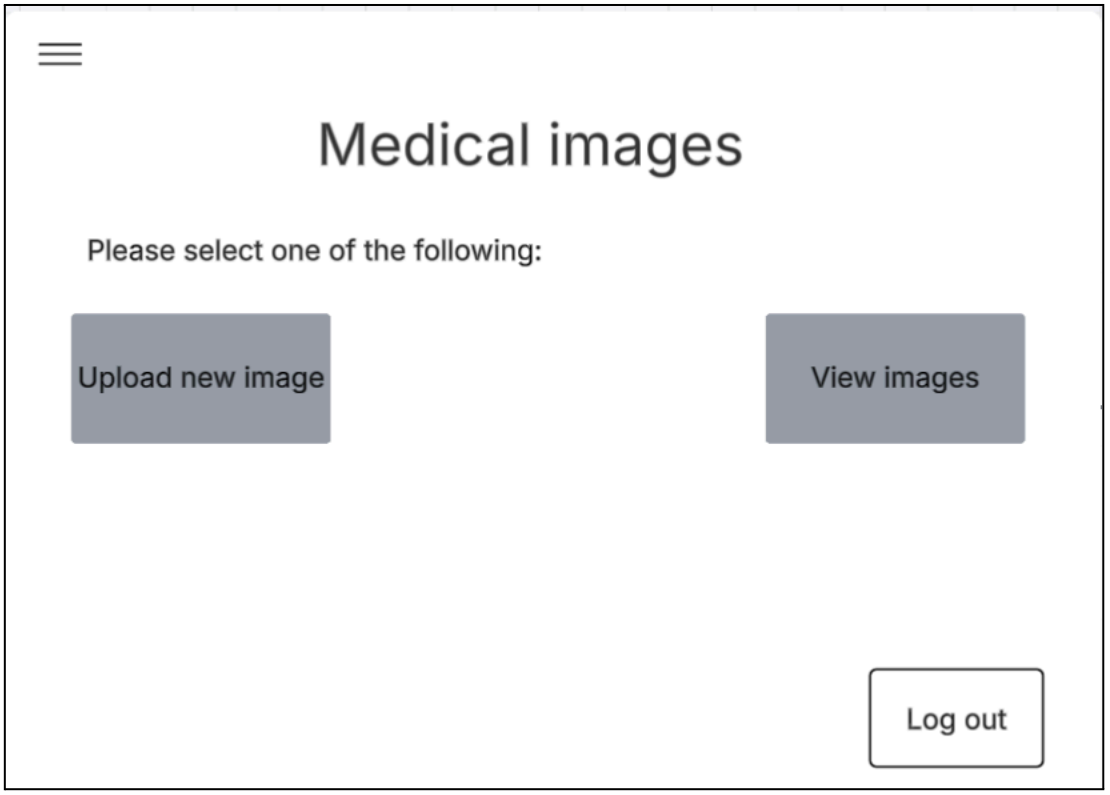
they aren't available, it will display what room they are based in currently, & what time they are next available.



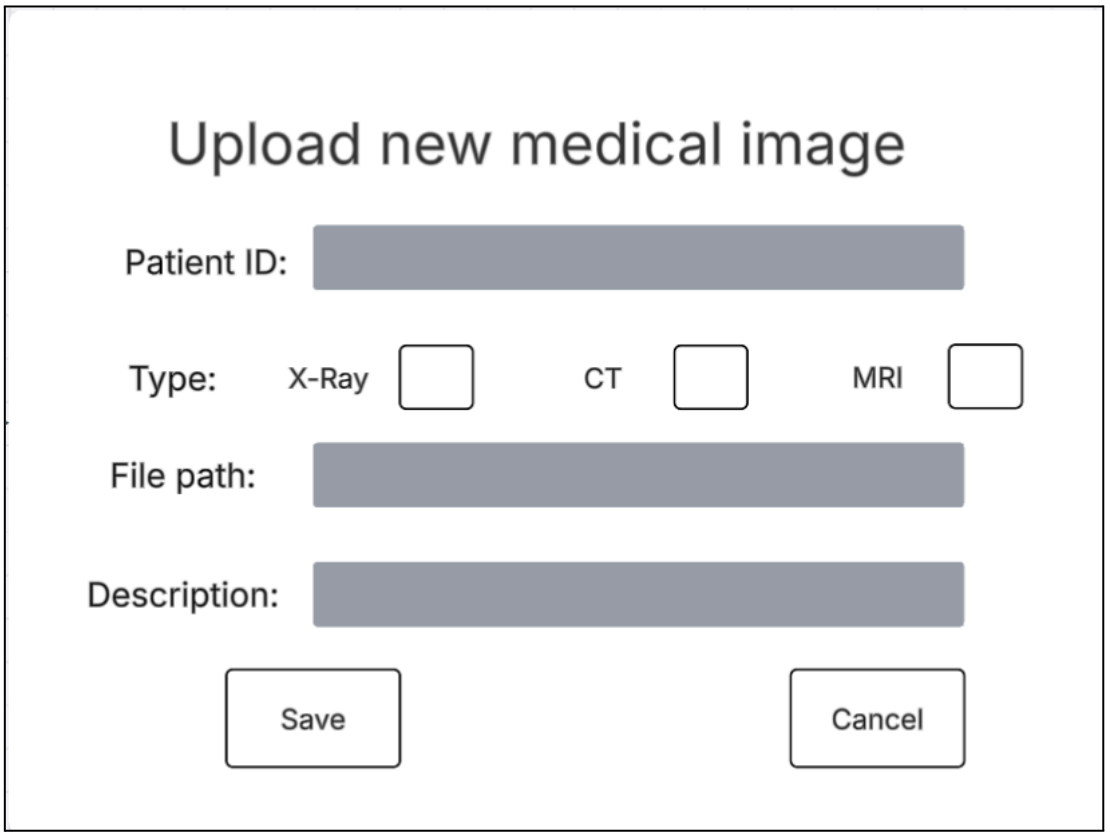
This is what Dr. Why will be able to see if he presses the staff availability button at the main menu. This component will be represented in table form with a coloured ovals, indicating whether staff are free or not. Red represents busy, whereas green represents available. On top of that, if they are busy, it will tell them which room is available or not.

In this wireframe, this is what users will see if previous users have entered the system, before the current user. They will be able to see all staff members who were logged on to the system, and display their status e.g. whether or not they are free, the room in which they are busy with, as well as the time in which they are next free/available and don't have duties. Furthermore, if staff members don't have a busy status (i.e, available), it will show empty spaces in the table. You may also notice that I added a (hamburger) menu icon in the top left corner, so that users can go back to the main menu as they please.

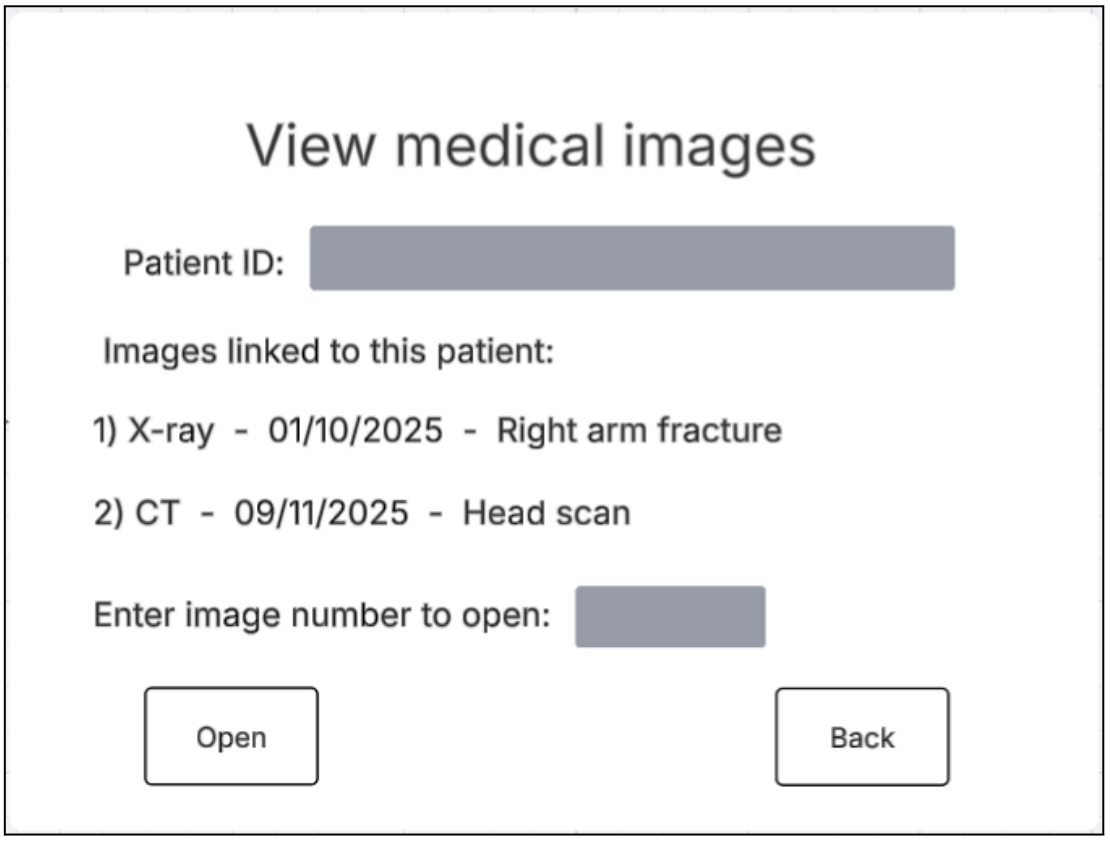
7) **Medical images function:**
   This function will allow you to access the medical images of patients registered on the system. This will be in 3 forms: X-ray scans, CT (Computed Tomography) scans, and MRI (Magnetic Resonance Imaging) scans. These images will be linked using patient IDs, just like in my ERD diagram, to identify which patients' image that correlates to.



This will be what users will see when they first access the medical images function/button in the main menu. This wireframe will allow staff to either upload new medical images, by entering the patient ID of a record, as well as the file path of the medical image. It will also allow staff to view images of a certain patient as well. I have also added a feature to log out as well as go back to the main menu using the hamburger icon at the top.



This is what users will see when they click on the “Upload new image” button on the previous wireframe. In this page, users can enter the patient ID of the record they would like to assign the image to. They will also be able to choose which type of medical image they would like to upload. Now, the key input in this image is the file path entry, where it will ask users for the file path of a certain image on the user's computer. Finally, it will then ask the user to input a description of the medical image e.g. right arm fracture.



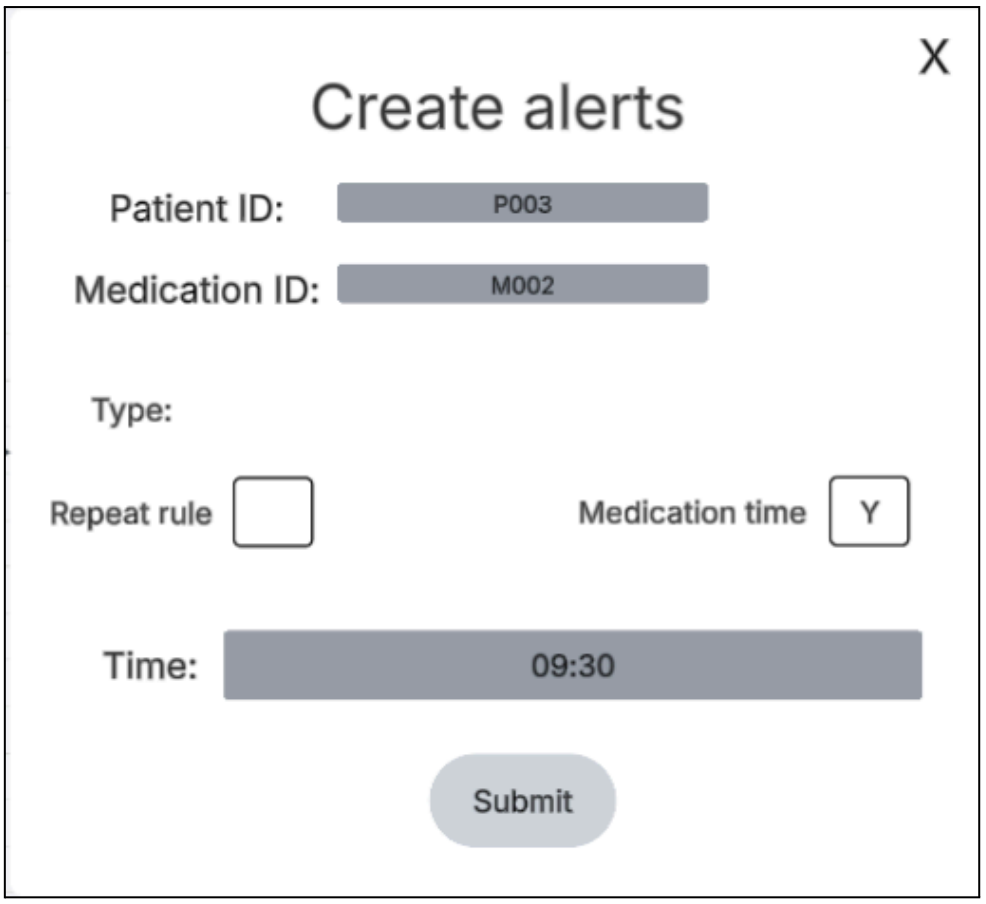
Now, then, this is what users will see when they click on the other option "View medical images". Firstly, it will ask them for the patient ID of the image. This is so that the options can be narrowed down if there are many medical images on the system, especially if one patient has multiple images (PATIENTS 1->∞ MEDICAL_IMAGES). And if a patient has multiple images, they can enter an image number to open the desired one. E.g. In this case, if the user wants to open the right arm fracture image, they would enter 1 in that space.

8) **Automated medication alert function**

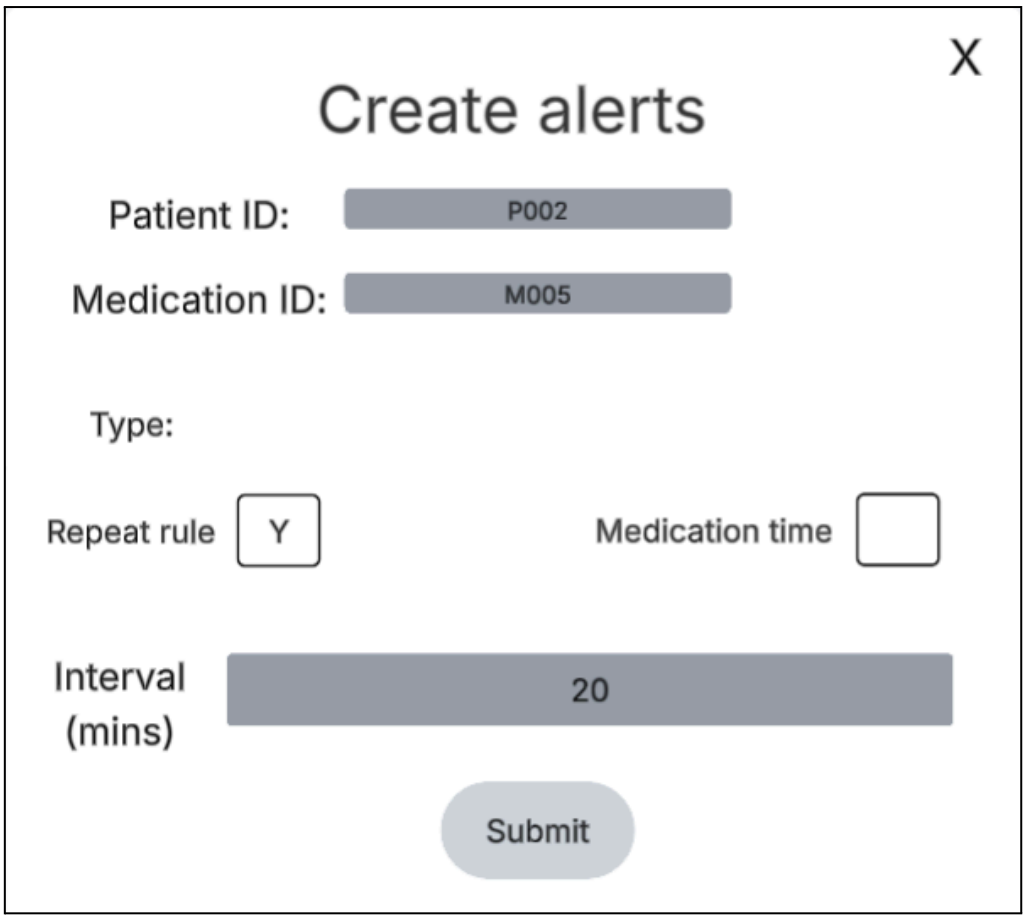
The wireframes in this function are what will bring the 2 functions together (i.e. the stock function and the EHR function). This will allow the user to create medication reminders, using the IDs of both the patient and the medication. Then, it will ask the user for a few options for how to set the alert. (1) it will either ask the user for a repeat rule where an alarm will sound in set time periods such as every 5 minutes. (2)Or, the user can choose a specific time for which the alert will be triggered such as 08:00 (however, there will be a specific time frame for which the user can create the alert from the users current time to the time 1 hour from then i.e. if users' current time is 04:30, alerts can be set from 04:30 to 05:30). (3) And lastly, the user can also create alerts by setting a timer for which at the end, the alarm will be triggered. The user can also be able to view medication alerts at the function of the menu.

I also plan to alert users in two ways… first is by a pop-up notification, showcasing that an alert is active/has occurred. Second, is by a sound alert, that when an alert is triggered, it will produce a semi-loud sound that users can hear through their computer.

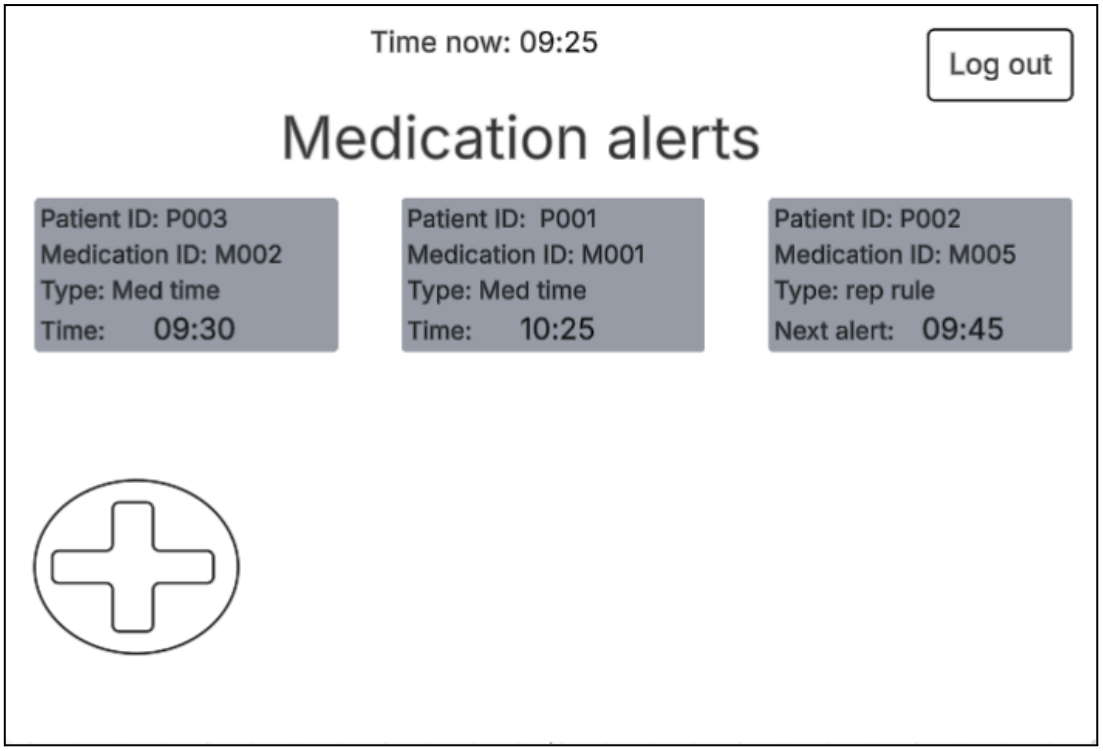
This will be what users will see when they access this feature for the first time. This is similar to the first wireframes where I added the plus symbol, so users can create alerts to remind them of medication times. I have also updated the wireframes to display the current time, and this will also be important in the next wireframe.



This is what users will see when they click on the plus symbol to create/add new alerts to remind staff to treat certain patients with medications. It will ask the user to input the ID of the patient, as well as the ID of the medication, so that they can both be linked. It will also ask the user what type of rule they would like to follow in order to alert the user e.g. using a repeat rule (repeat every x amount of minutes) or input a standard time for which the patient is treated. You may also notice that I input a time that is forward than the time in the last wireframe (e.g. 09:15 is the time in the last wireframe and 09:30 is still yet to come). When the user is finished filling out all of the information, they can press submit and create their alert.



Similarly, this is what users will see if they choose to input the repeat rule, instead of the standard time of medication. Instead of asking them to input the time, it will ask them to input the interval, in minutes. In this case, an input of 20 will mean that the alarm will sound once, every 20 minutes.



This wireframe shows what happens when multiple alerts are created. This is designed so that the wireframe displays the current time, as well as displays the medication times in bigger fonts so staff can keep track of future medication times. In summary, staff can use this function menu to now view future alerts and create new alerts.

This wireframe showcases how a pop-up notification will look like for medication alerts. This will be an alert that will show up in the centre of the screen no matter what part of the system the user is accessing. This pop-up will show information such as the name of the patient receiving medication, as well as the medication being delivered to

the patient. Finally, to close the pop-up, the user can either press the buttons to either administer the medication to the patient, or dismiss/close the pop-up. Either way, this will result in the pop-up disappearing from the screen.

END OF WIREFRAMES

3.2.2 d) What are the key variables, data structures, and classes that will be used in the solution and why will they be useful?

My system is an object-oriented hospital management system. For this reason, I will be using a different variety of classes, methods and attributes.. I will also be using a collection of data structures, such as hash tables (Dictionaries), relational tables, lists/arrays, and binary search trees

The main classes in my system will be:

- Medication
- Patient
- Room and Booking
- MedReminder
- ImageRef
- Staff and User

All of these classes will be based on my sub-modules in my decomposition diagram and the IPSO tables that define how the data will be stored, and processed.

1) Medication stock module

This module will be in charge of storing each medicine, tracking stock levels, detecting low stock, and resupplying when needed. This will be implemented, using a class with methods for adding and resupplying stock

1.1) Medication class - attributes and methods

This class will represent each medecine that the user will store and use for prescribing medication and setting up alerts. This will align with the attributes I have already described in my IPSO table for stock monitoring.

1.1.1) Attributes

Attribute	Type	Validation	Reason for inclusion

ID	string	Unique ID (auto-generated)	Used to link prescriptions, reminders and medication entries
name	String	Must be non-empty	Identifies the medicine in lists and alerts
category	String	Must be non-empty e.g. “painkiller”, “antibiotic”	Helps staff quickly understand the type of medicine
description	String	Optional	Allows extra information such as what symptoms it treats
maxStock	int	Positive integer and > 0	Upper limit for the stock bar
reorderLevel	int	0 < and < maxStock	Threshold for low stock warnings
currentQty	int	Kept in range: 0 < and < maxStock	Keeps track of current stock and low stock alerts

- ID allows the system to give each medication a unique name, for linking with other tables such as prescriptions, reminders and administration logs.
- maxStock, currentQty and reorderLevel directly drive the green stock bar and low-stock pop-ups in the GUI.
- Name and category are used for sorted lists and filtering, making it easier for staff to locate medicines quicker.

1.1.2) Methods

Methods	Parameters	Description	Reason for inclusion
addMedication()	Name, category, description, maxStock, reorderLevel	• Creates a new object, validates the fields and assigns a unique ID. • Set currentQty = maxStock initially. • Saves it to storage and returns the ID for the new record.	Used to link prescriptions, reminders and medication entries
generateId()	Name, category, description, maxStock, reorderLevel	• Generates an ID based on the order in which the medicine is entered into the system. • e.g. if its the first medicine entered, the ID will be M001	Ensures that each medication registered on the system has a different identifier for lookup.
resupplyMedication()	id, amount	• Looks up medication by ID • Increases currentQty by amount • Returns the message “Ok:	Used when stock is low or a delivery has arrived. This ensures that stock doesn’t run

		resupplied”	out
consumeMedication()	id, amount	• Used when medicine is administered to a patient • Decreases currentQty by amount • If there isn’t enough stock, return “Error: insufficient stock”	Keeps stock levels updated and in sync with medication administrations and helps prevents taking in too much stock than there is available
getMedication()	id	• Simple accessor to retrieve one medication object for display or editing	Used as a getter method that will access the attribute “”
listMedicationByName()	Name	• Loads all medications and returns them in a list sorted alphabetically by name	Used to provide a summary page in the main overview page of the functions’ GUI
listLowStock()	none	• Returns a list of medications where currentQty <= reorderLevel	Used to identify which medications the user may need to resupply in order to provide medication to patients.
suggestReorderQuantity ()	id	• Looks up a medication and returns maxStock - currentQty	Provides a sensible default amount when staff choose to restock, reducing input errors.

- isPositiveInteger(x): checks that values like maxstock and reorderLevel are sensible

1.2) Data structures (Medication Stock level):

1.2.1) Dictionary / Hash tables (main data structure):

Dictionaries are used for Pseudocode and writing Hash tables as Pseudocode. Hash tables are the python/HLL equivalent of the pseudocode dictionary.

What is it? = Hash tables are data structures that store key-value pairs and allow fast access to data using a hash function. This hash function converts the key into an index in an array. The key stores the unique identifier for the data, whereas the value stores the data associated with the key.

Key = id of the medication
Value = Medication object with all attributes
Reason = This gives very fast average lookup when the system needs to:

- Find a medication by id during an alert
- Update stock when medicine is consumed
- Resupply certain medicine

In summary, all medication objects are stored in a dictionary (**meds[id]**) which behaves as a hash table. This allows for fast lookup by ID whenever stock needs to be updated or displayed.

1.2.2) Lists for display/sorting (secondary data structure):

This list will be used alongside the listMedicationByName() method, which will return a list of all Medication objects. The list is then sorted by name.

Reason = Lists are appropriate when showing multiple medicines in the GUI and for applying simple sorting operations

1.3) Variables (Medication Stock level):

These variables will be the key parameters and local variables that appear inside the stock function, inside some of the methods

1.3.1) AddMedication()

Variable	Type	Scope	Initial value	Purpose
name	string	Parameter	Provided by user	Name of medication to be added
Category	String	Parameter	Provided by user	Category/type (antibiotic, painkiller, etc)
description	String	Parameter	Provided by user	Extra information about what the medicine is used for
maxStock	integer	Parameter	Provided by user	Upper limit for the medication stock level
reorderLevel	integer	Parameter	Provided by user	Threshold at which low-stock alerts will be triggered
m	Medication	Local	New instance	Temporary variable storing the new medication object before saving
id	string/integer	Local	From generateId()	Generated unique identifier returned to the user

These variables are mainly parameters provided by the user via the GUI. The m variable is a local instance of the Medication class, which is first validated, entered and then stored in the **meds** dictionary and in the MEDICATION_STOCK table

1.3.2) resupplyMedication()

Variable	Type	Scope	Initial value	Purpose
amount	integer	Parameter	Provided by user	Amount of stock to add
id	string	Parameter	Provided by user	Identifies which medication to fetch or resupply
m	Medication	Local	New instance	Holds the medication being resupplied

Local variable **m** is used to hold the current version of the medication object while its currentQty is
adjusted. The function then stores the updated object back into the **meds** dictionary and overwrites the corresponding row in the MEDICATION_STOCK table.

1.3.3) listLowStock()

Variable	Type	Scope	Purpose
all	List of medication	Local	Holds all medications loaded from storage
lows	List of medication	Local	Holds only medications that are smaller than or equal to the reorderLevel
list	List of medication	Local	Used when returning sorted medication by name

These local list variables are used to filter and sort medications for display. Returning lists is appropriate here because the data is going to be displayed in tables or menus rather than accessed by a single key.

2) EHR patient module

This module will be responsible for storing and managing patient information, e.g. personal details, medical history, conditions, contact details and relevant patient notes. It will allow staff to register new patients, update their records when circumstances

change, and view patient information when needed. This will be implemented using a Patient class, alongside functions for adding, editing, viewing, and deleting records

2.1) Patient class - attributes and methods

2.1.1) Attributes

Attribute	Type	Validation	Reason for inclusion
ID	string	Unique ID (auto-generated)	Used to link prescriptions, reminders and medication entries.
firstName	string	Should be non-empty (between 2 and 15 characters)	Used to identify patients on screen and in search results.
lastName	string	Should be non-empty (between 2 and 15 characters)	Main field used for sorting and searching the patient list so staff can find patients quickly.
email	string	Should match the basic email syntax e.g. "....@gmail.com" (lowercase)	Allows staff to contact patients and send reminders: making it lowercase makes searching more reliable.
phone	string	Digits only (between 8 and 15 characters)	Provides a quick way for staff to call patients or guardians if email is not accessible.
dob	date = string	Must be a valid date and not in the future	Used to confirm identity and to support age-related clinical decisions.
medicalConditions	string	Optional: must be a valid text string	Stores key current conditions of patients (e.g. asthma, diabetes) so staff can see important information in one place, instantly.
medications	string	Optional: must be a valid text string	Summary of the patients' used medication as a quick reference, alongside the medication module
notes	string	Optional: must be a valid text string	Allows extra information such as patient preferences or warnings that do not fit into other fields.

created_at	datetime = string	Automatically-set when record is created (immutable)	Used to record when the patient was first entered into the system for auditing purposes.
updated_at	datetime = string	Automatically-updated when record is edited (immutable)	Shows when the patient record was last changed, helping staff acknowledge that the records are up-to-date.

2.2.1) Methods

Methods	Parameters	Description	Reason for inclusion
addPatient()	firstName, lastName, email, phone, dob, medicalConditions, medications, notes, p, patients	• Creates a new patient, validates all fields and assigns a unique ID • Sets **created_at** and **updated_at** • Saves the information to storage	Main entry point for registering and entering new patients on the system.
generatePatientId()	p	• Generates a unique patient ID based on existing records (e.g. P001, P002, ...)	Ensures every patient can be referenced safely by other modules (bookings, images, medication, reminders)
updatePatient()	id, firstName, lastName, email, phone, dob, medicalConditions, medications, notes.	• Looks up the patient by ID, validates the new values and updates the stored record • Refreshes updated_at	Allows staff to correct or change patient details while keeping the same ID
viewPatient()	id	• Finds a single patient by ID and returns all their details on screen	Allows staff to open and access a patient's full electronic health record and information.
listPatientsByLastName()	none	• Loads all patients & sorts them by lastName and firstName • Returns an ordered list.	Provides a list of patients which is alphabetically ordered. This makes it easy for staff to browse records
searchPatientsByText()	queryText	• Searches across name, or email and returns a list of matching patients	Allows staff to search by either name or email, so they can find patients if they know part of a name or contact detail.
deletePatient()	id	• Locates the patient by ID and removes the record from the hash table	Allows records to be removed when created in error or when required by principles of the

2.2) Data structures (EHR - patient records)

2.2.1) Dictionary / Hash tables (main data structure)

Just like the medication stock level function, the main in-memory data structure for the EHR module will be a dictionary/hash table:

Key = id of the medication
Value = Patient object with all attributes (name, contact details, medical conditions, etc.)
Reason = Using a hash table means the system can very quickly:

- Find a patient by ID when staff open a record from the EHR summary screen.
- Update an existing patient when staff edit their details.
- Link other modules (room bookings, prescriptions, images, reminders) to the correct patient using the same ID.
- Enforces uniqueness of patient IDs so that records are not mixed up between different people.

In summary, all patient objects are stored in a hash table (patients[id]) which acts like a dictionary in pseudocode. This allows for fast lookup by ID whenever a patient's details need to be viewed, edited or linked to another part of the system..

2.2.2) Lists for display, sorting, and searching (secondary data structure)

Lists will be used as a secondary data structure for displaying and searching EHR records. They're created from the dictionary or loaded from storage when needed:

- **listPatientsByName ()** = Used for the main EHR overview screen, where staff can scroll through a list of patients. This list is alphabetically ordered and sorted by lastName and then firstName.
- **searchPatientsByText (queryText)** = Builds a temporary list, containing only patients whose name or email contains the search term. This list is then shown on the GUI as search results

Reason = lists are appropriate when:

- Showing **multiple patients** on screen at once (e.g. the EHR summary view).
- Applying simple operations like **sorting** (alphabetically by surname) and **filtering** based on a search query.
- The order of items matters to the user (e.g. A–Z ordering, or "top matches" list).

Using a dictionary gives fast lookup speeds using patient ID, whereas the lists sit on top for ordered display and searching in the user interface.

2.3) Key Variables (EHR - patient records)

These variables are required to store, manage, and validate patient information within the EHR module of the system. These variables allow hospital staff to accurately create, view, update, and delete patient records while ensuring that all data stored is consistent, valid, and up-to-date.

2.3.1) addPatient()

Variable	Type	Scope	Initial value	Purpose
firstName	string	Parameter	Provided by user	Patient's first name; validated by isNonEmptyString().
lastName	string	Parameter	Provided by user	Surname; main field used for identifying patients in a field & alphabetical sorting
email	string	Parameter	Provided by user	Corresponding contact email, checked by isValidEmail().
phone	string	Parameter	Provided by user	Phone number: validated as digits only with limits for the length (9-12 characters)
dob	date	Parameter	Provided by user	Date of Birth: checked by isValidDate
medicalConditions	string/list	Parameter	Optional user input	Allows a user to enter a patient's medical data e.g. symptoms & passed straight into the patient record
medications	string/list	Parameter	Optional user input	Entry field for entering what medications a patient takes or has taken before.
notes	string	Parameter	Optional user input	Field for adding additional information e.g. patient allergies
p	string	Local	Created inside function	Holds the patient object being added, to the system

patients	Dictionary	Global	Existing dictionary	Updated using patient[p.id] = p so that the record can be written to memory when fetching patients on he system

- This function (for adding patients) calls the isNonEmptyString, isValidEmail, isValidPhone and isValidDate before creating the record.
    - If any check fails, a clear error message will be returned e.g. "ERROR: invalid email", instead of saving erroneous data.

2.3.3) updatePatient()

Variable	Type	Scope	Initial value	Purpose
id	string	Parameter	Created by system	Holds the unique ID of the patient being updated, must already exist in the patients dictionary.
p	Patient	Local	Created by system	Holds the patient object being updated using the loadPatient(id). If the object exists, the object is updated and saved in memory.
All field parameters used in the addPatient() function	strings	Parameters	Provided by user	New values being provided by the user after editing the patient record.

- Function uses the same validation functions as the addPatient() functions to ensure data consistency.
- If the id is not found, it returns the message "ERROR: patient not found", which protects the user from accidentally creating an identical twin of the patient record, when editing.

3) Room and booking module

This module will be in charge of handling room bookings and ensuring that no more than one staff member can book the same rooms at the same time (conflict detection). This module also allows staff to choose how long each room will be booked for (maximum amount of time is 10 minutes), as well as which staff member booked which room.

3.1) Room class - attributes and methods

3.1.1) Room class attributes

Attribute	Type	Purpose	Validation	Reason for inclusion
roomLabel	string	Stores the name or label of the room (e.g. “Surgery room A”)	Cannot be empty	Used to identify specific rooms registered in the hospital system e.g. recovery bay 1, screening room 2, etc.
roomid	string	Uniquely identifies each hospital room	Creates a unique ID for each specific room	This allows staff to identify which rooms are booked or unbooked & ensure bookings reference the correct room.

These attributes define the rooms registered on the system, and assign a unique ID to each room so that bookingId, staffId, and patientId can be linked with the ID of the room that hospital staff members choose to book.

3.1.2) Room class methods:

Methods	Parameters	Description	Reason for inclusion
saveRoom()	room	Stores the room object in memory so that the room details are retained between sessions.	Required to ensure that hospital room information is not lost when the system is closed and reopened.
initDefaultRooms()	none	Creates a set of predefined hospital rooms when the system starts.	Allows the system to operate with a fixed, realistic set of rooms without requiring manual input.
listRooms()	none	Returns a list of all rooms currently registered in the system.	Enables staff to view all hospital rooms when checking availability or making bookings
isRoomAvailable()	roomID, startTime, endTime	Checks whether a room is free during a specific time period.	Prevents double booking and scheduling conflicts.
listAvailableRooms()	startTime, endTime	Returns all rooms that are available within a given time frame.	Improves usability by helping staff quickly identify suitable rooms.

3.1.1) Booking class attributes

Attribute	Type	Validation	Reason for inclusion
		Unique ID assigned to each booking, ID created depends on the order in which	Used as a foreign key in STAFF, PATIENTS, and ROOM tables.

id	string	rooms are booked (e.g. if all rooms are available to book and a user is the first to book a room, the id will be B001).	Helps staff identify how many rooms were booked, used for auditing.
roomId	string	Unique ID assigned to each registered room on the system.	Helps to identify specific rooms registered in the system, by assigning a unique ID to each specific room.
staffId	string	Unique ID assigned to each staff member/user upon signing up to the system. Usually determined by the order that staff enter the system (e.g. S001 if they're the first to enter the system)	Helps to identify which staff members booked a specific room and is identified using their firstName and staffId
patientId	string	Unique ID assigned to each patient upon entering their record. Usually determined by the order that staff enter the system (e.g. P001 if they're the first to enter the system)	Helps to identify which patient the room is booked for. (e.g. a room may be booked for patient P003, for a surgery).
start_At	string/integer (time)	Ensures that the start at value is accurate (e.g. if a user books a room at 01:30 PM, the value should display start at, 01:30 PM)	Used to set the start time of a certain booking
end_At	string/integer (time)	The maximum amount of time a user can book for each room is 10 minutes (if a user books a room for 6 minutes at 01:30 PM, the end at value should be 01:36 PM)	Used to identify when unavailable rooms will next be available for booking from other staff members and patients.

These attributes are responsible for managing the characteristics of each booking object of the class booking. These attributes outline when the booking will start and end, as well as which staff member is assigned to that room booking, and determine their availability.

3.1.2) Booking class methods:

Methods	Parameters	Description	Reason for inclusion
bookRoom()	roomId, staffId, patientId, startTime, endTime	This creates a new booking for a room if the inputted time period is valid and the room is available.	The most essential method in the whole module, ensures rooms are booked correctly and safely

isRoomAvailable()	roomId, starTime, endTime	This checks whether the room that the user wants to book is already booked during a certain time period	This ensures that rooms are not double booked and helps to prevent scheduling conflicts for users trying to book the same room.
overlaps() (maybe)	aStart(existing start), aEnd(existing end), bStart(new start), bEnd(new end)	This determines whether or not two time ranges for user room- bookings overlap with the same room.	This allows the system to accurately check and detect booking clashes.
listBookingsByRoom()	roomId	This returns all bookings that are associated with a certain room that can be made at different times.	This method allows staff to view the room's usage, as well as existing schedules/booking times (e.g. for when they could use the room next).
cancelBooking() (maybe)	bookingId	Removes an existing booking from the system	This method allows staff to cancel booking times if they accidentally booked the wrong room.
bookingDuration()	Booking :b	Calculates the duration of a booking	This method allows staff to see how long a room is booked for before being able to book the room themselves.

3.2) Data structures (room and booking module)

3.2.1) Dictionary / Hash tables (main data structure)

I have decided to create a dictionary/hash table for my room and booking classes, just like my other modules, such as the EHR module:

Rooms dictionary

Key = roomId / ID of the room
Value = Room object
Purpose = Stores all hospital rooms in memory
Reasons for inclusion = Using a dictionary / hash table for hospital rooms will allow for fast lookup when checking availability or creating bookings of rooms.

Booking dictionary

Key = bookingId / ID of the room
Value = Booking object
Purpose = This different dictionary stores all active room bookings that are registered on the system.

Reasons for inclusion = Using a different dictionary for bookings, allows for efficient validation when entering booking information (e.g. booking duration), as well as detecting conflicts between users booking the same room that is already booked for a certain time period.

3.2.2) Lists (secondary data structure)

Lists will be used as a secondary data structure to display room and booking objects in a sorted format, to the user.

roomList (list)

Value = Room objects
Purpose = Used to display all rooms to staff
Reason for inclusion = Using a list to display all registered rooms allows the rooms to be displayed in a set order and display multiple rooms to the user.

bookingList (list)

Value = Booking objects
Purpose = List used for displaying and sorting registered bookings
Reason for inclusion = Using a list allows the bookings to be sorted in terms of duration (e.g. shortest time to longest), room order (e.g. room1, room2, etc (see pseudocode)), or start time (e.g. when is the closest booking time).

availableRooms (list)

Value = Room objects
Purpose = List to temporarily store rooms available in a time window
Reason for inclusion = This list simplifies presenting availability results to the user, and shows the current available rooms the user is able to book.

In summary, all room and booking objects will be stored using a hash table. This will allow for fast lookup by roomId or bookingId for when users need to see available rooms, cancel bookings or actually book rooms for a certain time frame. Additionally, lists will also be used to display rooms alphabetically or show rooms in chronological order.

3.3) key Variables (room and booking module)

These key variables are defined to manage room allocations/bookings, validate booking times, as well as prevent conflicts between different bookings. I will also write about key variables used in this function, instead of writing about all variables defined in this module, to focus on the key variables and their purpose within this module.

Variable	Type	Scope	Initial value	Purpose

roomId	string	Parameter	Provided by system	This uniquely identifies the room being booked and used to link each booking to a specific hospital room.
bookingId	string	local	Provided by system	This stores the identifier for a booking once it has been successfully created, allowing bookings to be viewed or cancelled later.
staffId	string	Parameter	Provided by system	This identifies the staff member responsible for the booking, which also links to staff availability (when checking for available staff)
patientId	string	Parameter	Provided by system	This links a booking to a specific patient, allowing rooms to be associated with patient treatment or procedures.
startTime	datetime	Parameter	Provided by user	This stores the starting time of the booking, and is used for overlap checking and validation
endTime	datetime	Parameter	Provided by user	This stores the ending time of the booking, calculates booking duration, and detects conflicts between different bookings.
bookings	dictionary	Global	Empty at system start	This stores all room bookings using bookingId, which allows for fast lookups when checking for room bookings
rooms	dictionary	Global	Initialised at system start	This stores all hospital rooms and their details, and enabling availability checks
overlapDetected	boolean	local	False	Control variable used during validation to determine whether a booking clashes with an existing booking for the same room and time frame.
durationMinutes	integer	local	Provided by system during runtime	Variable used for validation and ensuring that the duration falls within the limits (1-15

				minutes) before the booking is confirmed

4) **Staff availability module**

This module will be in charge of determining whether hospital staff are available or unavailable during a specific time frame. This is done by analysing future room bookings and identifying which staff member is assigned to that booking. Staff availability will also be calculated dynamically, meaning that when a start_time of a booking occurs, that staff member will be notified, and their status changes to unavailable.

4.1) Staff class - attributes and methods

4.1.1) Staff class attributes

Attribute	Type	Purpose	Validation	Reason for inclusion
id	String	Uniquely identifies each staff member registered in the system. (e.g, "S001", "S002", etc)	Must be unique and system-generated	This Allows staff to be referenced when checking availability and linking to bookings.
firstName	String	Stores the staff member's first name.	Must be a non-empty string, limit between 1-35 characters	This is used for displaying staff information to users.
lastName	String	Stores the staff member's surname	Must be a non-empty string, limit between 1-35 characters	Used for identification, sorting, and display purposes
role	String	Stores the staff member's job role (e.g. nurse, surgeon).	Must be a non-empty string, limit between 1-35 characters	Allows users to distinguish between different types of staff when viewing availability

These attributes are used to define users/staff members registered on the system, by storing a unique ID along with their name and role. The staff ID allows each member of staff to be uniquely referenced across the system, especially when staff are linked to room bookings and availability checks.

4.1.2) Staff class methods

Methods	Parameters	Description	Reason for inclusion
isValidDateTime() (NTS: This will be the on the same day)	t : datetime	Checks whether the value that the user enters is a valid DateTime, when checking if	This ensures that availability checks are only performed using valid time values

		staff are available within a certain time period.	
getActualEnd()	b : Booking	This returns the end time of a booking.	This allows staff members to see when another staff member is next free or when their assigned booking ends.
isTimeBetween()	startAt, endAt, t	Determines whether a specified time occurs during an existing booking.	This is a helper function used for detecting whether a staff member is busy
isStaffBusyAt()	staffId, t	Checks whether a staff member has an active booking at a given time.	This method is used to determine staff availability dynamically
listAvailableStaffAt()	t	Returns a list of staff members who are not busy at a specified time (set to available current staff on default)	Allows users to quickly identify which staff are available
listUnavailableStaffAt()	t	Returns a list of staff who are currently busy	This also allows users to see which staff members are unavailable during a certain period.

These methods are used to determine staff availability by analysing room bookings with a specific point in time. This module will also calculate availability dynamically rather than storing it. This is to ensure that staff status is always accurate and up-to-date.

4.2) Data structures (Staff availability module)

4.2.1) Dictionary / Hash tables (main data structure)

For the main data structure, I have decided to use dictionaries / hash tables for the staff availability, just like the other modules in the system such as the Room & booking and EHR modules.

Staff dictionary

Key = staffID
Value = Staff object
Purpose = Stores all staff members registered on the system in memory
Reasons for inclusion = Using a dictionary/hash table for staff allows for fast lookup of staff information when checking if a specific staff member is available or seeing which staff members are busy or free during a given time frame.

Booking dictionary (referenced)
Purpose = This analyses existing room bookings to determine whether a staff member is currently assigned to a booking at a given time.
Reasons for inclusion = Reusing the existing booking avoids duplication of date and ensures staff availability is always calculated using up-to-date booking information.

4.2.2) Lists (Secondary data structure)
These lists will be used to display available staff and unavailable staff to the user, in a chronological order.

availableStaff (list)
Value = Staff objects/staff IDs
Purpose = Used to store and display all staff members who are available during a specific time period.
Reason for inclusion = Using a list allows the system to store staff members who are not currently assigned to a booking. This list will also be regenerated each time availability is checked, which ensures that data on staff availability is up-to-date.

unavailableStaff (list)
Value = Staff objects/staff IDs
Purpose = Used to store and display all staff members who are currently busy or assigned to a room booking.
Reason for inclusion = Using a list makes it easier to group unavailable staff, due to existing bookings. This list will also support sorting or filtering (e.g. by name or next available space), allowing staff to identify which staff members are unavailable during a specific period of time.

In summary, using a hash table for staff members allows for fast lookup by staffId to determine whether that staff member is assigned to a booking and hence if they're available or not. I have also referenced the booking dictionary, as this will allow the system to determine whether a staff member is available or not, by checking current bookings.

4.3) Key variables (staff availability module)

These key variables are used to check and display whether staff members are available at a given time, based on existing bookings. Availability will be calculated automatically rather than stored. This ensures that information on staff availability is always up to date.

Variable	Type	Scope	Initial value	Purpose

staffId	string	Parameter	Provided by system	Identifies the specific staff member whose availability is being checked.
checkTime	DateTime	Parameter	Provided by user	This variable represents the date and time at which staff availability is evaluated.
staff	Dictionary	Global	Existing data	Stores all staff records, & allows the system to iterate through all staff members & return their availability status.
bookings	Dictionary	Global	Existing data	Stores all room bookings records and used to determine if a staff member is assigned to a booking.
availableStaff	List	Local	Empty list	Used to temporarily store staff members who are not assigned to a booking at a given time.
unavailableStaff	List	Local	Empty list	Used to temporarily store staff members who are assigned to a booking at a given time.
isBooked	Boolean	Local	false	Used to determine whether a staff member has an overlapping booking during the availability check.

5) Medical images module

This module allows staff to upload, store, and view medical images that are linked to patient records, such as x-ray, CT and MRI scans. Images are associated with patients using unique IDs, to ensure images are linked to the correct patient. This module provides staff with quick access to imaging data, & support diagnosis and treatment decisions.

5.1) ImageRef class - attributes & methods

5.1.1) ImageRef class attributes

Attribute	Type	Purpose	Validation	Reason for inclusion

imageId	String	Uniquely identifies each medical image registered on the system (e.g. I001, I002,I003, etc).	Must be unique and system-generated	Required to distinguish each image and allow precise retrieval.
patientId	String	Links the image to a specific patient	Must exist in patient records	This ensures images are always associated with the correct patient.
type	String	Stores the image type (x-ray, CT, MRI)	Must either be: "x-ray", "CT", "MRI"	Allows the system to categorise images and filter by scan type
filePath	String	Stores the file location of the image	Must not be empty	This is required to actually open and display the image from the computer
takenAt	DateTime	Stores when the image was taken	Must be a valid date/time from user	This allows staff to check when an image was taken, to determine whether the image shows up to date information.
uploadedAt	DateTime	Stores when the image was uploaded	Automatically generated	Allows tracking of when images were added to the system.
notes	String	Optional description of the image	Optional field	This allows staff to add useful medical notes (e.g. fracture location)

These attributes are used to represent a medical image and link it to the correct patient record using a unique ID. These also allow the image type, file path, and timestamps to be stored, and ensures that the images can be accurately identified. The attribute "notes" is an optional input for providing additional medical information about the patient.

5.1.2) ImageRef class methods

Methods	Parameters	Description	Reason for inclusion
uploadImage()	patientId, type, filePath, takenAt, notes	This creates a new medical images record and links it to a patient	Allows staff to enter new medical images on the system
viewImage()	imageId	Opens and displays the selected medical image	Enables staff to view images for diagnostics and review

listImagesByPatient()	patientId	Returns all images linked to a specific patient	Allows staff to view a patient's full imaging
listImagesByType()	patientId, type	Returns images of a specific scan type for a patient	Helps staff filter images by image mode (e.g. MRI)
getLatestImages()	patientId, limit	Returns the most recent images for a patient	Allows quick access to the most recent diagnostic data/medical images.

These methods allow staff to upload, retrieve, and view medical images linked to patient records. By supporting filtering by patient and image type, the system will able to provide quick access to images/diagnostic data. This will ensure imaging data is managed efficiently whilst being accurate.

5.2) Data structures (Medical images module)

5.2.1) Dictionary / Hash table (main data structure)

For the main data structure, I have decided to use a dictionary / hash table for storing the medical images objects, just like the other modules e.g. the EHR system

Images dictionary

Key = imageId
Value = ImageRef object
Purpose = Dictionary used to store and manage all medical images registered on the system, with each image linked to unique image Id.
Reason for inclusion = A dictionary allows each medical image to be accessed quickly using it's unique image Id. This will allow for fast lookups, when viewing or retrieving images, and makes it easier to link images to patient records.

5.2.2) List (Secondary data structure)

This list will be used to display the medical images and be able to sort these images in terms of type, patient, or id.

imageList (list)

Value = ImageRef objects
Purpose = Used to temporarily store and display collections of medical images, such as all images for a specific patient or images of a particular scan type.
Reason for inclusion = Using a list allows medical images to be grouped together and displayed in a clear order. This allows for sorting and filtering of images, and makes it easier for staff to browse images by categories (e.g. patient or type).

In summary, using a dictionary to store medical images, allows the system to quickly retrieve image records using unique IDs. Temporary lists are then generated from this dictionary to filter and sort images by patient or scan type.

5.3) Key variables (Medical images module)

These key variables are used to manage the storage, retrieval, and display of medical images that are linked to patient records. Images are accessed using unique identifiers and filtered dynamically when needed.

Variable	Type	Scope	Initial value	Purpose
imageId	String	Parameter	Passed into function	This identifies the specific medical image to be retrieved or viewed.
patientId	String	Parameter	Passed into function	This links the images to a specific patient record for retrieving specific images.
type	String	Parameter	Passed into function	This specifies the scan type (e.g. X-ray, CT, MRI) used for filtering images
images	Dictionary	Global	Existing data	This stores all medical image records using unique image IDs for fast lookup
result	list	Local	Empty list	This is used to temporarily store filtered images (e.g. by patient or type) for display.
filePath	String	Local	Retrieval value	This stores the file location of the image to allow it to be opened and displayed
limit	Integer	Parameter	Passed into function	This restricts the number of images returned when displaying recent scans.

6) Medication reminders module

This module is responsible for automatically alerting staff when patients are due to receive medication. It links patients, medications, and schedules times to ensure doses are given correctly and on time. Reminders are generated in real time using pop-up notifications and sound alerts, which helps reduce the chance of missed medication.

6.1) MedReminder class - attributes and methods

6.1.1) MedReminder class attributes

Attribute	Type	Purpose	Validation	Reason for inclusion
reminderId	String	This uniquely identifies each medication reminder registered (e.g. “R001”, “R002”)	Must be unique and system generated	This is required to reference, update, and acknowledge individual reminders.
patientId	String	This links the reminder to a specific patient	Provided by user, must match an existing patient ID	This ensures medication reminders apply to the correct patient.
medicationId	String	This identifies the medication to be given to the patient	Provided by user, must match an existing medication ID	This is required to ensure correct medication delivery and stock updates.
doseAmount	Integer	This stores the dosage amount to be given to patients	Must be a positive integer	This is used to ensure that the correct dosage is given to the patient
scheduleType	String	This defines the scheduling rule used for that patient (e.g. fixed time, every … hours)	Must be one of the supported schedule types. (e.g. “fixed times”, “interval values”)	This allows for different medication scheduling patterns, to match the medication needs of patients.
scheduleData	String	This stores the schedule specific details (e.g. times of day, or interval times.)	Must match the format required by scheduleType	This is used to provide flexibility for unique medication patterns to cater to each patient’s medical needs.
nextDueAt	DateTime	This stores the next time the medication is due	Must be a valid future dateTime	This is used to trigger reminders dynamically in real time
active	Boolean	This is used to indicate whether the reminder is occurring right now	Must be true or false	This allows reminders to be stopped when treatment ends
createdAt	DateTime	This records when the reminder was created	Must be an accurate timestamp	This helps staff track when the reminder was created.

These attributes store all information required to generate and manage medication reminders in real time. By combining scheduleType and scheduleData, the system is able to support flexible medication reminder patterns, whilst reducing the module’s overall complexity. Using unique IDs for reminderId ensures accurate linking between patients, medications and reminders.

6.1.2) MedReminder class methods

Methods	Parameters	Description	Reason for inclusion
createReminderTimeOfDay()	patientId, medicationId, doseAmount	This is used to create a medication reminder that triggers at specific times of the day.	This will be the main entry point allowing staff to input reminders into the system.
createReminderEveryXMins()	patientId, medicationId, doseAmount, everyMins, startAt	This creates a medication reminder that repeats every X amount of minutes.	This allows for a flexible interval based medication schedule, where staff can automatically create repeat reminders without having to input the reminder into the system.
showPopup()	patientId, medicationId, doseAmount, nextDueAt	This is used to display an on-screen pop-up notification showing the patient, medication and dosage when a medication is due.	This allows staff to be given clear visual context so they know exactly which medication needs to be administered and which patient it needs to be given to.
playSound()	none	This is used to play an audible alert when a medication reminder becomes due	This ensures that staff are alerted even if they're not actively looking at the screen.
computeNextDue()	r : reminder, now	This is used to calculate the next time a medication dose is due based on the reminder's schedule.	This allows staff to see how much time there is left until the next reminder.
listActiveReminders()	none	This returns all active medication reminders sorted by next due time	This allows hospital staff to keep track of up and coming reminders in the near future.
listRemindersForPatients()	patientId	This returns all active medication reminders that are due for a specific patient.	This allows hospital staff to view a patient's medication schedule e.g. future medication times
listRemindersDueAt()	now	This is used to identify reminders that are due at the current time.	This is used to trigger alerts only when medication is actually due now.
reminderTick()	now	This checks for due reminders and trigger alerts	By periodically checking for due reminders, this acts as one of the most vital time-based control loop for the reminder module.

editReminder()	reminderId, doseAmount, scheduleType, ScheduleData	This is used to update an existing reminder and recalculates the next due time	This allows staff to update an existing reminder, in case a staff member entered the wrong timings, or assigned the wrong medication or patients to that reminder.
deactivateReminder()	reminderId	This disables the medication reminder when staff click on the pop-up notification	This allows staff to disable reminders once treatment is complete e.g. patients have already been given medication.

These methods handle how medication reminders are created, checked, and updated over time. Reminder times are calculated using simple time rules(e.g. repeat rule, every X minutes, etc) rather than storing lots of future reminders. The reminderTick method also regularly checks for due reminders and triggers alerts at the correct time.

6.2) Data structures (Medication reminders module)

6.2.1) CSV / Relational table (main data structure)

For the main data structure, I have decided to use a CSV file, unlike most of the data structures in my other modules. This will be used to permanently store the reminders objects even when the system is down.

Reminders CSV file

Fields = reminderId, patientId, doseAmount, scheduleType, ScheduleData, nextDueAt, active, createdAt
Purpose = The CSV file is used to store all medication reminders persistently, ensuring that scheduled medication times remain available even when the system is turned off.
Reason for inclusion = Medication reminders are time-based, so they must remain and persist when the system is turned off, to avoid missed doses. This is why using a CSV file makes it easier to store reminder data reliably after system shut down.

6.2.2) Dictionary / Hash tables (Secondary data structure)

For the second data structure, I have decided to use a dictionary / hash table for storing the medication reminders objects of the MedReminder class, just like the other modules e.g. the EHR module.

Reminders dictionary

Key = reminderId
Value = MedReminder object
Purpose = The dictionary is used to store and manage all active medication reminders in the system, with each reminder linked to a unique reminder ID.
Reason for inclusion = Using a dictionary allows medication reminders to be accessed quickly using their unique reminder ID. This allows for fast lookups when editing

medication schedules, or deactivating reminders and ensures that reminders can be reliably linked to patients and medications.

6.2.3) Lists (secondary data structure)

This list will be used as a temporary data structure to store medication reminders in terms of specific categories e.g. closest upcoming reminder or schedule type.

reminderList (list)

Value = MedReminder objects
Purpose = This is used to temporarily store and display a collection of reminders, such as all active reminders or reminders for a specific patient.
Reason for inclusion = Using a list to store medication reminders, allows them to be easily sorted and/or filtered based on different conditions e.g. nextDueAt or scheduleType. This makes it easier for staff to view and manage reminders that are related by similar conditions.

In summary, medication reminders are stored permanently using a CSV file ensures that medication schedules are not lost at system restart. Using a dictionary allows for fast access and updates to reminder and schedule data. Temporary lists are generated from the dictionary when reminders need to be displayed in a specific order or category.

6.3) key variables (Medication reminders module)

These key variables are used to manage the scheduling and activation of medication reminders. These variables control when reminders are turned on, how often the medication is administered, and whether a reminder is currently active.

Variable	Type	Scope	Initial value	Purpose
remindersFile	String	Global	reminders.csv	This is used to store the file path of the CSV file used for permanent storage of medication reminders.
reminders	Dictionary	Global	{}	This dictionary stores all medication reminder objects in memory.
reminderList	List	Local	[]	This is used to temporarily store reminders for sorting and displaying reminder data based on certain categories.

currentTime	DateTime	Local	nowTimestamp()	Used to hold the current system time when checking which reminders are due.
dueReminders	List	Local	[]	This temporarily stores reminders that are due to be triggered at the current time.
snoozeMinutes	Integer	Constant	5 (minutes)	This is used to define how long a reminder is delayed for when the user selects snooze. E.g. 5 minutes.

7) Account system and log in module

This module will be responsible for managing user authentication and controlling user acces within the system. This module will ensure that only authorised staff members can log in and use the system's features by validating usernames and passwords, using a cryptographic hash function. This means that no passwords are stored in plain text, and so this helps protect sensitive patient and staff data from unauthorised access.

7.1) User & session classes - attributes and methods

7.1.1) User class attributes

Attribute	Type	Purpose	Validation	Reason for inclusion
userId	String	This uniquely identifies each user registered on the system (e.g., "U001", "U002", etc)	Must be unique and system generated	Required to identify users in the order in which they registered onto the system.
username	String	This is used to store the unique login identifier for the user	Must not be empty and must be unique	This is required to identify registered users and authenticate logins
salt	String	This stores the unique salt used during password hashing	Must be non-empty and randomly generated	By adding an additional string to the password before its hashed, this will improve the password's security.
passwordHash	String	This stores the cryptographic hash of the salted password	Must only be generated via the hash function.	This is used to ensure that passwords are not stored in plain text
createdAt	DateTime	This records when the user account was created	Auto-generated timestamp	This allows hospital staff to see when a user's account was created.

lastLoginAt	DateTime	This stores the time of the last successful login before the current user	Updated on successful login and/or sign-up	This allows for monitoring of activity on a user's account.
active	Boolean	This indicates whether the account is enabled or disabled.	Must be TRUE or FALSE	This allows accounts to be disabled without deleting data

The User class stores all information required for secure authentication and log in. Passwords are protected, using a cryptographic hash faunction with salting, which ensures that sensitive data is never stored in plain text. Time stamps also support security monitoring by staff members.

7.1.2) User class methods

Methods	Parameters	Description	Reason for inclusion
createUser()	Username, password	Creates a new user account by creating a salt, hashing the password and storing the user correctly,	This means that staff do not have to re make a new user account, each time they log-in and log-off the system
verifyPassword()	Username, password	This is used to hash the input password with the stored salt and compares it to the original hashed values.	Used when staff login or change passwords to validate credentials.
login()	Username, password	This authenticates the user and creates a new session if credentials match with the original hash values stored.	This is required to let staff re-access the system once they have logged on before.
loginWithExistingStaffAuthorisation()	existingUsername, existingPassword, newUsername, newPassword	This creates a new user only if an existing staff member authorises the action.	This prevents unathorised users from creating their account to potentially steal or edit patient records.
changePassword()	Username, oldPassword, newPassword	This give staff the option to change their password by generating a new salt and hash.	This allows users to securely update their passwords, whether if their password is compromised or forgotten.
loadUser()	username	This retrieves the user record from storage using the username.	This is required to access the user data for authentication and updates to credentials.
generateSalt()	none	This generates a random salt value that is added to a password before hashing.	This means that multiple staff members are able to have same or similar password, as the salt

			ensures that similar passwords produce different hash values.
hashPassword()	Password, salt	This produces a cryptographic hash from the password and salt.	This ensures that passwords are never stored or compared in plain text.

The user class methods are responsible for account creation, authenticating staff login information and password management. These methods manage passwords by using a salted cryptographic hash function to protect sensitive data. Additional methods are also created to update credential information.

7.1.1) Session class attributes

Attribute	Type	Purpose	Validation	Reason for inclusion
token	String	This stores the unique identifier of each session	Must be randomly generated and unique	Used to keep track of user sessions on the same computer.
username	String	This identifies which user that session belongs to.	User input must link to an existing username	Usernames are used to link user accounts directly to sessions
createdAt	DateTime	This records when the session was created e.g. what time the user logs on	Auto-generated timestamp	This helps staff see which staff members registered and what time they logged on.
expiresAt	DateTime	This records when the session has expired e.g. what time the user logs off	Auto-generated timestamp, must be after createdAt time.	This is used to evaluate when a staff's session becomes invalid.
active	Boolean	This is used to indicate whether the session is still currently valid.	Must be TRUE or FALSE.	This allows sessions to be revoked.

The session class is responsible for managing users after they logged into the system. By linking a secure token, each session is linked to a user and has a defined expiry date where the session becomes no longer available. Sessions can be invalidated without deleting records, which helps with safe logout and retaining data on staff credentials.

7.1.2) Session class methods

Methods	Parameters	Description	Reason for inclusion
loadSession()	token	This retrieves a session using its unique token.	This is required to validate all active sessions.

logout()	sessionToken	This is used to deactivate an existing session.	This allows users to securely end their session.
isSessionValid()	sessionToken	This checks whether a session exists, is active, and has not expired.	This prevents unauthorised access via expired sessions.
generateId()	None	This generates a unique session token	This ensures that each session is uniquely identifiable.
nowTimestamp()	None	This method is used to return the current system time.	This is used for session expiry and for staff to check the current time.

These session class methods are responsible for managing use sessions after login. These sessions are validated using tokens, expiry/finish times, and active flag status (true or false) to reduce security risks. Logout and expiry handling ensure that sessions cannot be reused indefinitely.

7.2) Data structures (Account system and log in module)

7.2.1) Dictionary / Hash table (main data structure)

For the main data structure, I have decided to use a dictionary/hash table, which will act as a cryptographic hash table to store the users login data securely.

Users dictionary (actual hash function)

Key = username
Value = user object
Purpose = This stores all user accounts that are registered on the system, as well as storing salted and hashed password data.
Reason for inclusion = Using a dictionary allows for fast lookups of user accounts by username, which is essential for login authentication. Storing only cryptographic hash values and salts ensures that passwords are never stored in plain text and so this improves the security of the system.

Session dictionary

Key = sessionId
Value = session object
Purpose = This stores all active and recently created login sessions, and links each session token to a specific user and session expiry times.
Reason for inclusion = Using a dictionary will allow session data to be accessed quickly using their unique session token. This allows for fast validation of sessions and supports secure logout by deactivating sessions.

In summary, this module uses dictionaries to manage user accounts and login sessions. User credentials are stored using salted and hashed values to protect sensitive data, while sessions are tracked using unique tokens and expiry times.

7.3) Key variables (Account system and log in module)

These key variables are used to manage user validation and session handling. These variables help with logins, and creating and validating sessions during system use. These variables also help track login activity and control whether user accounts or session are active. All together, these variables ensure controlled and secure access to the system.

Variable	Type	Scope	Initial value	Purpose
users	Dictionary	Global	Empty	This stores all user accounts and information e.g. username, and salted and hashed password data.
sessions	Dictionary	Global	Empty	This stores all active and recently created login sessions, which is indexed by the session token.
sessionDurationMins	Integer	Constant	15 minutes	This is used to define how long a session remains valid before it expires.
currentTimestamp	DateTime	Local	Current time = nowTimestamp()	This is used to compare expiry times of the session and update login activity.
generatedSalt	String	Local	Provided by system	This stores the unique salt added to passwords, during user creation or changes to user passwords.
generatedToken	String	Local	Provided by system	This is used to temporarily store a unique session token, when creating a new session

3.2.3) Describe the approach to testing

3.2.3 a) What is the test data to be used during the iterative and post development phases of the project

There will be a wide range of test data used in the system, to ensure that the system works up to expected standards and unauthorises certain invalid entries. Testing will be in the form of Iterative testing (testing occurs continuously at each part of the cycle). Below I have produced a range of test plans for iterative development testing, for each part of my system. I have also decided to document the iterative testing data in this document, and then document the post-development testing data, during the development phase of the project. This is because, this system will be developed using the agile methodology of software development, where testing will take place as each part of the system is being developed. Whereas, post-development testing will take place, after the whole program is created.

1) **Account system and login module**

Iterative testing plan:

Test number	Description	Test data & type of data	Expected Outcome	Outcome (After part is designed)	Reason for including test
1	User enters a valid password with at least 8 characters for the password	**Valid:** "HelloWorld" > 8 characters	Password will be accepted and hashed before being stored.		This test ensures that valid passwords meeting the security rules are accepted by the system.
2	User logs in with correct username and correct password	**Valid:** Username = "staff002" Password = "HelloWorld32"	Login is successful and the session token is created.		This confirms that the authentication works correctly and for correct credentials.
3	User attempts login with correct username but incorrect password	**Invalid:** Username = "staff001" Password = "Hello_World367"	Login is denied and no session token is created.		This ensures unauthorised access is prevented, even when usernames are valid.
4	User enters a password shorter than 8 characters	**Invalid:** "ABC123"	Password is rejected, and an error message is returned.		This prevents weak passwords from being entered into the system.
	The user enters a password exactly at the minimum length	**Boundary:** "abcd1234"	The password must be accepted and hashed as		This ensures that users can create passwords that have exactly 8

5	boundary e.g. 8 characters.	(8 characters)	normal.		characters, and can correctly handle boundary data.
6	Two similar passwords are hashed and compared,	**Boundary:** "HelloWorld123" and "HelloWorld1234"	Hash values produced are completely different		This ensures that the smallest change in a user's password produces an entirely different hash value.

2) Medication stock module

Iterative testing plan

Test number	Description	Test data & type of data	Expected Outcome	Outcome (After part is designed)	Reason for including test
1	Staff adds a new medication with valid stock and reorder levels.	**Valid:** Name = "Paracetomol" MaxStock = 100 ReorderLevel = 20	Medication is created successfully and stored with current stock, auto set to max stock .e.g. CurrentStock = 100		This ensures that valid medication data is accepted and stored correctly.
2	Medication stock is reduced when medication is administered (given to patients).	**Valid:** MedicationID = "M001" amountConsumed = 1	Stock level is reduced by 1 and updated in the system, or by amount.		This ensures that stock decreases correctly during medication use
3	User attempts to add medication with a negative maximum stock value	**Invalid:** MaxStock = -10 ReorderLevel = 8	Medication is rejected and an error message is returned.		This prevents invalid values being entered for the stock limits, which could break stock calculations
4	User attempts to resupply medication beyond its maximum stock	**Invalid:** CurrentStock = 95 ResupplyAmount = 20	Resupply attempt is rejected and an error message is returned because stock level is capped at the max stock value.		This ensures that stock levels cannot exceed realistic levels i.e. the max stock level.
5	Medication stock exactly reaches the reorder threshold.	**Boundary:** CurrentStock = 30 ReorderLevel = 30	Medication is flagged as "low stock" and warnings will be sent to the user,		This verifies correct behaviour when stock reaches the reorder boundary.
	Medication stock is reduced to	**Boundary:**	Stock level becomes 0 and		This ensures that the system

6	zero	CurrentStock = 1 AmountConsumed = 1	further consumption is prevented		can handle boundary values without allowing for negative stock.

3) **Patient EHR module**

Iterative testing plan

Test number	Description	Test data & type of data	Expected Outcome	Outcome (After part is designed)	Reason for including test
1	Staff creates a new patient record using valid details.	**Valid:** FirstName = "John" Surname = "Smith" Email = "JohnSmith@bemail.com" Phone = 07652 467967 DOB = 19/07/1998	Patient record is successfully created and stored, with a unique PatientID		This confirms that only valid patient data is accepted and stored correctly
2	Staff updates and existing patient's contact information	**Valid:** PatientID = "P001" Phone = 07652 467967	Patient record is successfully updated and saved.		This ensures that existing records can be safely modified without affecting other patient records.
3	Staff tries to create a patient record, with missing required fields.	**Invalid:** FirstName = "" Surname = "Smith"	The patient record should be rejected and an error message is displayed to the user.		This prevents incomplete patient records from being stored.
4	Staff enters an invalid email when creating a patient record.	**Invalid:** Email = "paulsmith.com"	Patient record is rejected (due to invalid email format) and an error message is displayed.		This ensures basic data validation of emails, which could lead to wrong contact information.
5	Staff enters a phone number at the minimum valid length (7 to 12 digits).	**Boundary:** Phone number = 0765644 (7 digits)	Phone number should be accepted as valid.		This ensures that boundary values are accepted for validation of phone numbers.
6	Staff deletes the last remaining patient record in the system.	**Boundary:** PatientID = "P001" -> only patient record stored	Patient record is removed and the system should be able to handle an empty patient list.		This tests the system's behaviour when the patient database transitions from one record to zero.

4) **Room & booking module**

Iterative testing plan:

Test number	Description	Test data & type of data	Expected Outcome	Outcome (After part is designed)	Reason for including test
1	Staff books an available hospital room with valid start and end times. (each room can only be booked for a maximum of 30 mins)	**Valid:** RoomID = "R001" Start = 10:00 End = 10:06	The booking should be successfully created and stored.		This checks that rooms can be booked normally when all details are valid.
2	Staff views all available rooms for a given time period.	**Valid:** Time range = 10:10-13:00	Only rooms not already booked during this time are shown.		This ensures that availability is shown and displayed correctly based on existing bookings.
3	Staff attempts to book a room that is already booked for at the same time.	**Invalid:** RoomID = "R001" Start = 10:00 End = 10:15	Booking should be rejected and an error message is shown and displayed		This prevents two bookings from using the same room at the same time.
4	Staff enters an end time that is before the start time.	**Invalid:** Start = 10:00 End = 08:40	Booking is rejected due to the invalid time range.		This stops logically incorrect bookings from being created.
5	Staff books a room, where the new booking starts exactly when the previous one ends with the same room.	**Boundary:** Existing = 10:00 - 10:10 New = 10:10 - 10:17	Booking is accepted as there's no time overlap.		Checks that the system handles edge cases where times meet exactly.
6	Staff books the last available room during a busy time slot.	**Boundary:** RoomID = "R005" (only free room) Time = 11:20 - 11:29	Booking is successful and no rooms remain available		This tests the system's behaviour when availability reaches its minimum.
7	Staff books a room for exactly the maximum amount of time (30 minutes)	**Boundary:** RoomID = "R003" Start = 03:10	Booking is accepted as it's still in between the upper time limit		This tests the system's behaviour when booking duration reaches the

		End = 03:40			maximum time frame.

5) **Staff availability module**

Iterative testing plan:

Test number	Description	Test data & type of data	Expected Outcome	Outcome (After part is designed)	Reason for including test
1	The system displays staff as available when they have no active room bookings.	**Valid:** StaffID = "S001" Current time = 10:00 Bookings = none	Staff member is shown as available		This checks that staff are correctly marked as available when free.
2	The system displays staff as unavailable during an active room booking.	**Valid:** StaffID = "S002" Booking = 09:00 - 09:20 Current time = 09:10	The staff member is shown as unavailable.		This ensures that staff are marked as busy when they're assigned to a room.
3	The system checks availability using an invalid/non-existent StaffID.	**Invalid:** StaffID = "S99"	The staff member is not found and availability is not shown.		This prevents availability being shown for staff that do not exist.
4	The system checks availability using an invalid time format.	**Invalid:** Time = 25:62	Availability check fails and an error is shown and displayed to the user.		This stops invalid time values from affecting availability logic.
5	Booking ends exactly at the current time.	**Boundary:** Booking = 09:30-10:00 Current time = 10:00	The staff member will be shown as available.		This checks correct behaviour when a booking has just ended.
6	Staff member is the only available person at the current time.	**Boundary:** StaffID = "S003" is free All other staff are booked.	Staff member is shown as available.		This tests the system's behaviour when availability reaches its minimum.

6) **Medical images module**

Iterative testing plan:

Test number	Description	Test data & type of data	Expected Outcome	Outcome (After part is designed)	Reason for including test
1	Staff uploads a valid medical image for an existing patient.	**Valid:** PatientID = "P001" File = "xray1.png" Type = X-ray	Image is uploaded and linked to the correct patient record.		Checks that valid images can be uploaded and stored correctly.
2	Staff views medical images linked to a patient record	**Valid:** PatientID = "P001"	All images linked to the patient are displayed.		This ensures that stored images can be retrieved and viewed when needed.
3	Staff attempts to upload an unsupported file type	**Invalid:** File = "image.pdf"	Upload is rejected and an error message is shown.		This prevents non-image files from being stored as medical images.
4	Staff attempts to upload an image for a non-existent patient	**Invalid:** PatientID = "P006" File = "Scan.png"	Upload is rejected and an error message is shown.		This stops images being linked to patients that do not exist.
5	Staff uploads an image that is exactly at the maximum allowed file size (5 MB)	**Boundary:** File size = "5 MB" (max limit)	Image is accepted and uploaded successfully		This checks the correct handling of image size limits.
6	Staff attempts to view medical images for a patient with no images stored	**Boundary:** PatientID = "P002" Images linked = 0	System returns a message e.g. "no images available".		This checks how the system behaves when no images exist.

7) **Medication reminders module:**

Iterative testing plan:

Test number	Description	Test data & type of data	Expected Outcome	Outcome (After part is designed)	Reason for including test
	The system triggers medication	**Valid:**	Pop-up notification and sound		This checks that reminders

1	reminders at the correct scheduled time.	PatientID = “P001” Medication =“Paracetomol” Time = 10:00 Current time = 10:00	alert are triggered.		trigger correctly at the scheduled time.
2	Staff acknowledges a medication reminder	**Valid:** Reminder acknowledged by StaffID = “S001”	Reminders are removed and medication stock is reduced.		This ensures that reminders can be completed and stored correctly.
3	The system attempts to trigger a reminder for a medication that is not scheduled.	**Invalid:** PatientID = “P002” No reminder set	No alert is triggered as there is no medication alert that was set for that patient		This allows false or unnecessary reminders to be prevented from being entered into the system.
4	The system attempts to load reminder data from a missing or invalid CSV file.	**Invalid:** CSV file is missing	System handles the error without crashing, as a result of inputting an invalid patient into the system.		This test ensures that the system can handle file-related errors in a safe way.
5	Reminder is scheduled exactly at the current time of the system.	**Boundary:** Reminder time = 10:00 Current time = 10:00	The reminder will trigger immediately		This checks for correct behaviour when scheduled time equals the current time.
6	Patient has no medication reminders scheduled.	**Boundary:** PatientID = “P003” Reminders set = 0	The system displays no reminders and continues running normally. E.g. no errors		This tests the system behaviour when no reminder data exists.

End of design document